

---

Gareth James • Daniela Witten • Trevor Hastie  
Robert Tibshirani • Jonathan Taylor

# An Introduction to Statistical Learning

with Applications in Python

First Printing: July 5, 2023

*To our parents:*

*Alison and Michael James*

*Chiara Nappi and Edward Witten*

*Valerie and Patrick Hastie*

*Vera and Sami Tibshirani*

*John and Brenda Taylor*

*and to our families:*

*Michael, Daniel, and Catherine*

*Tessa, Theo, Otto, and Ari*

*Samantha, Timothy, and Lynda*

*Charlie, Ryan, Julie, and Cheryl*

*Lee-Ann and Isobel*

# Preface

Statistical learning refers to a set of tools for *making sense of complex datasets*. In recent years, we have seen a staggering increase in the scale and scope of data collection across virtually all areas of science and industry. As a result, statistical learning has become a critical toolkit for anyone who wishes to understand data — and as more and more of today’s jobs involve data, this means that statistical learning is fast becoming a critical toolkit for *everyone*.

One of the first books on statistical learning — *The Elements of Statistical Learning* (ESL, by Hastie, Tibshirani, and Friedman) — was published in 2001, with a second edition in 2009. ESL has become a popular text not only in statistics but also in related fields. One of the reasons for ESL’s popularity is its relatively accessible style. But ESL is best-suited for individuals with advanced training in the mathematical sciences.

*An Introduction to Statistical Learning, With Applications in R* (ISLR) — first published in 2013, with a second edition in 2021 — arose from the clear need for a broader and less technical treatment of the key topics in statistical learning. In addition to a review of linear regression, ISLR covers many of today’s most important statistical and machine learning approaches, including resampling, sparse methods for classification and regression, generalized additive models, tree-based methods, support vector machines, deep learning, survival analysis, clustering, and multiple testing.

Since it was published in 2013, ISLR has become a mainstay of undergraduate and graduate classrooms worldwide, as well as an important reference book for data scientists. One of the keys to its success has been that, beginning with Chapter 2, each chapter contains an **R** lab illustrating how to implement the statistical learning methods seen in that chapter, providing the reader with valuable hands-on experience.

However, in recent years **Python** has become an increasingly popular language for data science, and there has been increasing demand for a **Python-**

based alternative to ISLR. Hence, this book, *An Introduction to Statistical Learning, With Applications in Python* (ISLP), covers the same materials as ISLR but with labs implemented in **Python** — a feat accomplished by the addition of a new co-author, Jonathan Taylor. Several of the labs make use of the **ISLP Python** package, which we have written to facilitate carrying out the statistical learning methods covered in each chapter in **Python**. These labs will be useful both for **Python** novices, as well as experienced users.

The intention behind ISLP (and ISLR) is to concentrate more on the applications of the methods and less on the mathematical details, so it is appropriate for advanced undergraduates or master’s students in statistics or related quantitative fields, or for individuals in other disciplines who wish to use statistical learning tools to analyze their data. It can be used as a textbook for a course spanning two semesters.

We are grateful to these readers for providing valuable comments on the first edition of ISLR: Pallavi Basu, Alexandra Chouldechova, Patrick Danaher, Will Fithian, Luella Fu, Sam Gross, Max Grazier G’Sell, Courtney Paulson, Xinghao Qiao, Elisa Sheng, Noah Simon, Kean Ming Tan, Xin Lu Tan. We thank these readers for helpful input on the second edition of ISLR: Alan Agresti, Iain Carmichael, Yiqun Chen, Erin Craig, Daisy Ding, Lucy Gao, Ismael Lemhadri, Bryan Martin, Anna Neufeld, Geoff Tims, Carsten Voelkmann, Steve Yadlowsky, and James Zou. We are immensely grateful to Balasubramanian “Naras” Narasimhan for his assistance on both ISLR and ISLP.

It has been an honor and a privilege for us to see the considerable impact that ISLR has had on the way in which statistical learning is practiced, both in and out of the academic setting. We hope that this new **Python** edition will continue to give today’s and tomorrow’s applied statisticians and data scientists the tools they need for success in a data-driven world.

*It’s tough to make predictions, especially about the future.*

-Yogi Berra

# Contents

|                                                                                         |            |
|-----------------------------------------------------------------------------------------|------------|
| <b>Preface</b>                                                                          | <b>vii</b> |
| <b>1 Introduction</b>                                                                   | <b>1</b>   |
| <b>2 Statistical Learning</b>                                                           | <b>15</b>  |
| 2.1 What Is Statistical Learning? . . . . .                                             | 15         |
| 2.1.1 Why Estimate $f$ ? . . . . .                                                      | 17         |
| 2.1.2 How Do We Estimate $f$ ? . . . . .                                                | 20         |
| 2.1.3 The Trade-Off Between Prediction Accuracy<br>and Model Interpretability . . . . . | 23         |
| 2.1.4 Supervised Versus Unsupervised Learning . . . . .                                 | 25         |
| 2.1.5 Regression Versus Classification Problems . . . . .                               | 27         |
| 2.2 Assessing Model Accuracy . . . . .                                                  | 27         |
| 2.2.1 Measuring the Quality of Fit . . . . .                                            | 28         |
| 2.2.2 The Bias-Variance Trade-Off . . . . .                                             | 31         |
| 2.2.3 The Classification Setting . . . . .                                              | 34         |
| 2.3 Lab: Introduction to Python . . . . .                                               | 40         |
| 2.3.1 Getting Started . . . . .                                                         | 40         |
| 2.3.2 Basic Commands . . . . .                                                          | 40         |
| 2.3.3 Introduction to Numerical Python . . . . .                                        | 42         |
| 2.3.4 Graphics . . . . .                                                                | 48         |
| 2.3.5 Sequences and Slice Notation . . . . .                                            | 51         |
| 2.3.6 Indexing Data . . . . .                                                           | 51         |
| 2.3.7 Loading Data . . . . .                                                            | 55         |
| 2.3.8 For Loops . . . . .                                                               | 59         |
| 2.3.9 Additional Graphical and Numerical Summaries . .                                  | 61         |
| 2.4 Exercises . . . . .                                                                 | 63         |
| <b>3 Linear Regression</b>                                                              | <b>69</b>  |
| 3.1 Simple Linear Regression . . . . .                                                  | 70         |
| 3.1.1 Estimating the Coefficients . . . . .                                             | 71         |
| 3.1.2 Assessing the Accuracy of the Coefficient<br>Estimates . . . . .                  | 72         |
| 3.1.3 Assessing the Accuracy of the Model . . . . .                                     | 77         |
| 3.2 Multiple Linear Regression . . . . .                                                | 80         |
| 3.2.1 Estimating the Regression Coefficients . . . . .                                  | 81         |

|          |                                                                          |            |
|----------|--------------------------------------------------------------------------|------------|
| 3.2.2    | Some Important Questions . . . . .                                       | 83         |
| 3.3      | Other Considerations in the Regression Model . . . . .                   | 91         |
| 3.3.1    | Qualitative Predictors . . . . .                                         | 91         |
| 3.3.2    | Extensions of the Linear Model . . . . .                                 | 94         |
| 3.3.3    | Potential Problems . . . . .                                             | 100        |
| 3.4      | The Marketing Plan . . . . .                                             | 109        |
| 3.5      | Comparison of Linear Regression with $K$ -Nearest<br>Neighbors . . . . . | 111        |
| 3.6      | Lab: Linear Regression . . . . .                                         | 116        |
| 3.6.1    | Importing packages . . . . .                                             | 116        |
| 3.6.2    | Simple Linear Regression . . . . .                                       | 117        |
| 3.6.3    | Multiple Linear Regression . . . . .                                     | 122        |
| 3.6.4    | Multivariate Goodness of Fit . . . . .                                   | 123        |
| 3.6.5    | Interaction Terms . . . . .                                              | 124        |
| 3.6.6    | Non-linear Transformations of the Predictors . . . . .                   | 125        |
| 3.6.7    | Qualitative Predictors . . . . .                                         | 126        |
| 3.7      | Exercises . . . . .                                                      | 127        |
| <b>4</b> | <b>Classification</b>                                                    | <b>135</b> |
| 4.1      | An Overview of Classification . . . . .                                  | 135        |
| 4.2      | Why Not Linear Regression? . . . . .                                     | 136        |
| 4.3      | Logistic Regression . . . . .                                            | 138        |
| 4.3.1    | The Logistic Model . . . . .                                             | 139        |
| 4.3.2    | Estimating the Regression Coefficients . . . . .                         | 140        |
| 4.3.3    | Making Predictions . . . . .                                             | 141        |
| 4.3.4    | Multiple Logistic Regression . . . . .                                   | 142        |
| 4.3.5    | Multinomial Logistic Regression . . . . .                                | 144        |
| 4.4      | Generative Models for Classification . . . . .                           | 146        |
| 4.4.1    | Linear Discriminant Analysis for $p = 1$ . . . . .                       | 147        |
| 4.4.2    | Linear Discriminant Analysis for $p > 1$ . . . . .                       | 150        |
| 4.4.3    | Quadratic Discriminant Analysis . . . . .                                | 156        |
| 4.4.4    | Naive Bayes . . . . .                                                    | 158        |
| 4.5      | A Comparison of Classification Methods . . . . .                         | 161        |
| 4.5.1    | An Analytical Comparison . . . . .                                       | 161        |
| 4.5.2    | An Empirical Comparison . . . . .                                        | 164        |
| 4.6      | Generalized Linear Models . . . . .                                      | 167        |
| 4.6.1    | Linear Regression on the Bikeshare Data . . . . .                        | 167        |
| 4.6.2    | Poisson Regression on the Bikeshare Data . . . . .                       | 169        |
| 4.6.3    | Generalized Linear Models in Greater Generality . . . . .                | 172        |
| 4.7      | Lab: Logistic Regression, LDA, QDA, and KNN . . . . .                    | 173        |
| 4.7.1    | The Stock Market Data . . . . .                                          | 173        |
| 4.7.2    | Logistic Regression . . . . .                                            | 174        |
| 4.7.3    | Linear Discriminant Analysis . . . . .                                   | 179        |
| 4.7.4    | Quadratic Discriminant Analysis . . . . .                                | 181        |
| 4.7.5    | Naive Bayes . . . . .                                                    | 182        |
| 4.7.6    | $K$ -Nearest Neighbors . . . . .                                         | 183        |
| 4.7.7    | Linear and Poisson Regression on the Bikeshare Data . . . . .            | 188        |
| 4.8      | Exercises . . . . .                                                      | 193        |

|          |                                                                     |            |
|----------|---------------------------------------------------------------------|------------|
| <b>5</b> | <b>Resampling Methods</b>                                           | <b>201</b> |
| 5.1      | Cross-Validation . . . . .                                          | 202        |
| 5.1.1    | The Validation Set Approach . . . . .                               | 202        |
| 5.1.2    | Leave-One-Out Cross-Validation . . . . .                            | 204        |
| 5.1.3    | $k$ -Fold Cross-Validation . . . . .                                | 206        |
| 5.1.4    | Bias-Variance Trade-Off for $k$ -Fold<br>Cross-Validation . . . . . | 208        |
| 5.1.5    | Cross-Validation on Classification Problems . . . . .               | 209        |
| 5.2      | The Bootstrap . . . . .                                             | 212        |
| 5.3      | Lab: Cross-Validation and the Bootstrap . . . . .                   | 215        |
| 5.3.1    | The Validation Set Approach . . . . .                               | 216        |
| 5.3.2    | Cross-Validation . . . . .                                          | 217        |
| 5.3.3    | The Bootstrap . . . . .                                             | 220        |
| 5.4      | Exercises . . . . .                                                 | 224        |
| <b>6</b> | <b>Linear Model Selection and Regularization</b>                    | <b>229</b> |
| 6.1      | Subset Selection . . . . .                                          | 231        |
| 6.1.1    | Best Subset Selection . . . . .                                     | 231        |
| 6.1.2    | Stepwise Selection . . . . .                                        | 233        |
| 6.1.3    | Choosing the Optimal Model . . . . .                                | 235        |
| 6.2      | Shrinkage Methods . . . . .                                         | 240        |
| 6.2.1    | Ridge Regression . . . . .                                          | 240        |
| 6.2.2    | The Lasso . . . . .                                                 | 244        |
| 6.2.3    | Selecting the Tuning Parameter . . . . .                            | 252        |
| 6.3      | Dimension Reduction Methods . . . . .                               | 253        |
| 6.3.1    | Principal Components Regression . . . . .                           | 254        |
| 6.3.2    | Partial Least Squares . . . . .                                     | 260        |
| 6.4      | Considerations in High Dimensions . . . . .                         | 262        |
| 6.4.1    | High-Dimensional Data . . . . .                                     | 262        |
| 6.4.2    | What Goes Wrong in High Dimensions? . . . . .                       | 263        |
| 6.4.3    | Regression in High Dimensions . . . . .                             | 265        |
| 6.4.4    | Interpreting Results in High Dimensions . . . . .                   | 266        |
| 6.5      | Lab: Linear Models and Regularization Methods . . . . .             | 267        |
| 6.5.1    | Subset Selection Methods . . . . .                                  | 268        |
| 6.5.2    | Ridge Regression and the Lasso . . . . .                            | 273        |
| 6.5.3    | PCR and PLS Regression . . . . .                                    | 280        |
| 6.6      | Exercises . . . . .                                                 | 283        |
| <b>7</b> | <b>Moving Beyond Linearity</b>                                      | <b>289</b> |
| 7.1      | Polynomial Regression . . . . .                                     | 290        |
| 7.2      | Step Functions . . . . .                                            | 292        |
| 7.3      | Basis Functions . . . . .                                           | 293        |
| 7.4      | Regression Splines . . . . .                                        | 294        |
| 7.4.1    | Piecewise Polynomials . . . . .                                     | 294        |
| 7.4.2    | Constraints and Splines . . . . .                                   | 296        |
| 7.4.3    | The Spline Basis Representation . . . . .                           | 296        |
| 7.4.4    | Choosing the Number and Locations<br>of the Knots . . . . .         | 297        |
| 7.4.5    | Comparison to Polynomial Regression . . . . .                       | 299        |

|          |                                                                                        |            |
|----------|----------------------------------------------------------------------------------------|------------|
| 7.5      | Smoothing Splines . . . . .                                                            | 300        |
| 7.5.1    | An Overview of Smoothing Splines . . . . .                                             | 300        |
| 7.5.2    | Choosing the Smoothing Parameter $\lambda$ . . . . .                                   | 301        |
| 7.6      | Local Regression . . . . .                                                             | 303        |
| 7.7      | Generalized Additive Models . . . . .                                                  | 305        |
| 7.7.1    | GAMs for Regression Problems . . . . .                                                 | 306        |
| 7.7.2    | GAMs for Classification Problems . . . . .                                             | 308        |
| 7.8      | Lab: Non-Linear Modeling . . . . .                                                     | 309        |
| 7.8.1    | Polynomial Regression and Step Functions . . . . .                                     | 310        |
| 7.8.2    | Splines . . . . .                                                                      | 315        |
| 7.8.3    | Smoothing Splines and GAMs . . . . .                                                   | 317        |
| 7.8.4    | Local Regression . . . . .                                                             | 324        |
| 7.9      | Exercises . . . . .                                                                    | 325        |
| <b>8</b> | <b>Tree-Based Methods</b>                                                              | <b>331</b> |
| 8.1      | The Basics of Decision Trees . . . . .                                                 | 331        |
| 8.1.1    | Regression Trees . . . . .                                                             | 331        |
| 8.1.2    | Classification Trees . . . . .                                                         | 337        |
| 8.1.3    | Trees Versus Linear Models . . . . .                                                   | 341        |
| 8.1.4    | Advantages and Disadvantages of Trees . . . . .                                        | 341        |
| 8.2      | Bagging, Random Forests, Boosting, and Bayesian Additive<br>Regression Trees . . . . . | 343        |
| 8.2.1    | Bagging . . . . .                                                                      | 343        |
| 8.2.2    | Random Forests . . . . .                                                               | 346        |
| 8.2.3    | Boosting . . . . .                                                                     | 347        |
| 8.2.4    | Bayesian Additive Regression Trees . . . . .                                           | 350        |
| 8.2.5    | Summary of Tree Ensemble Methods . . . . .                                             | 353        |
| 8.3      | Lab: Tree-Based Methods . . . . .                                                      | 354        |
| 8.3.1    | Fitting Classification Trees . . . . .                                                 | 355        |
| 8.3.2    | Fitting Regression Trees . . . . .                                                     | 358        |
| 8.3.3    | Bagging and Random Forests . . . . .                                                   | 360        |
| 8.3.4    | Boosting . . . . .                                                                     | 361        |
| 8.3.5    | Bayesian Additive Regression Trees . . . . .                                           | 362        |
| 8.4      | Exercises . . . . .                                                                    | 363        |
| <b>9</b> | <b>Support Vector Machines</b>                                                         | <b>367</b> |
| 9.1      | Maximal Margin Classifier . . . . .                                                    | 367        |
| 9.1.1    | What Is a Hyperplane? . . . . .                                                        | 368        |
| 9.1.2    | Classification Using a Separating Hyperplane . . . . .                                 | 368        |
| 9.1.3    | The Maximal Margin Classifier . . . . .                                                | 370        |
| 9.1.4    | Construction of the Maximal Margin Classifier . . . . .                                | 372        |
| 9.1.5    | The Non-separable Case . . . . .                                                       | 372        |
| 9.2      | Support Vector Classifiers . . . . .                                                   | 373        |
| 9.2.1    | Overview of the Support Vector Classifier . . . . .                                    | 373        |
| 9.2.2    | Details of the Support Vector Classifier . . . . .                                     | 374        |
| 9.3      | Support Vector Machines . . . . .                                                      | 377        |
| 9.3.1    | Classification with Non-Linear Decision<br>Boundaries . . . . .                        | 378        |
| 9.3.2    | The Support Vector Machine . . . . .                                                   | 379        |



|           |                                                      |            |
|-----------|------------------------------------------------------|------------|
| 9.3.3     | An Application to the Heart Disease Data . . . . .   | 382        |
| 9.4       | SVMs with More than Two Classes . . . . .            | 383        |
| 9.4.1     | One-Versus-One Classification . . . . .              | 384        |
| 9.4.2     | One-Versus-All Classification . . . . .              | 384        |
| 9.5       | Relationship to Logistic Regression . . . . .        | 384        |
| 9.6       | Lab: Support Vector Machines . . . . .               | 387        |
| 9.6.1     | Support Vector Classifier . . . . .                  | 387        |
| 9.6.2     | Support Vector Machine . . . . .                     | 390        |
| 9.6.3     | ROC Curves . . . . .                                 | 392        |
| 9.6.4     | SVM with Multiple Classes . . . . .                  | 393        |
| 9.6.5     | Application to Gene Expression Data . . . . .        | 394        |
| 9.7       | Exercises . . . . .                                  | 395        |
| <b>10</b> | <b>Deep Learning</b>                                 | <b>399</b> |
| 10.1      | Single Layer Neural Networks . . . . .               | 400        |
| 10.2      | Multilayer Neural Networks . . . . .                 | 402        |
| 10.3      | Convolutional Neural Networks . . . . .              | 406        |
| 10.3.1    | Convolution Layers . . . . .                         | 407        |
| 10.3.2    | Pooling Layers . . . . .                             | 410        |
| 10.3.3    | Architecture of a Convolutional Neural Network . .   | 410        |
| 10.3.4    | Data Augmentation . . . . .                          | 411        |
| 10.3.5    | Results Using a Pretrained Classifier . . . . .      | 412        |
| 10.4      | Document Classification . . . . .                    | 413        |
| 10.5      | Recurrent Neural Networks . . . . .                  | 416        |
| 10.5.1    | Sequential Models for Document Classification . .    | 418        |
| 10.5.2    | Time Series Forecasting . . . . .                    | 420        |
| 10.5.3    | Summary of RNNs . . . . .                            | 424        |
| 10.6      | When to Use Deep Learning . . . . .                  | 425        |
| 10.7      | Fitting a Neural Network . . . . .                   | 427        |
| 10.7.1    | Backpropagation . . . . .                            | 428        |
| 10.7.2    | Regularization and Stochastic Gradient Descent . .   | 429        |
| 10.7.3    | Dropout Learning . . . . .                           | 431        |
| 10.7.4    | Network Tuning . . . . .                             | 431        |
| 10.8      | Interpolation and Double Descent . . . . .           | 432        |
| 10.9      | Lab: Deep Learning . . . . .                         | 435        |
| 10.9.1    | Single Layer Network on Hitters Data . . . . .       | 437        |
| 10.9.2    | Multilayer Network on the MNIST Digit Data . .       | 444        |
| 10.9.3    | Convolutional Neural Networks . . . . .              | 448        |
| 10.9.4    | Using Pretrained CNN Models . . . . .                | 452        |
| 10.9.5    | IMDB Document Classification . . . . .               | 454        |
| 10.9.6    | Recurrent Neural Networks . . . . .                  | 458        |
| 10.10     | Exercises . . . . .                                  | 465        |
| <b>11</b> | <b>Survival Analysis and Censored Data</b>           | <b>469</b> |
| 11.1      | Survival and Censoring Times . . . . .               | 470        |
| 11.2      | A Closer Look at Censoring . . . . .                 | 470        |
| 11.3      | The Kaplan–Meier Survival Curve . . . . .            | 472        |
| 11.4      | The Log-Rank Test . . . . .                          | 474        |
| 11.5      | Regression Models With a Survival Response . . . . . | 476        |

|           |                                                    |            |
|-----------|----------------------------------------------------|------------|
| 11.5.1    | The Hazard Function . . . . .                      | 476        |
| 11.5.2    | Proportional Hazards . . . . .                     | 478        |
| 11.5.3    | Example: Brain Cancer Data . . . . .               | 482        |
| 11.5.4    | Example: Publication Data . . . . .                | 482        |
| 11.6      | Shrinkage for the Cox Model . . . . .              | 484        |
| 11.7      | Additional Topics . . . . .                        | 486        |
| 11.7.1    | Area Under the Curve for Survival Analysis . . . . | 486        |
| 11.7.2    | Choice of Time Scale . . . . .                     | 487        |
| 11.7.3    | Time-Dependent Covariates . . . . .                | 488        |
| 11.7.4    | Checking the Proportional Hazards Assumption . .   | 488        |
| 11.7.5    | Survival Trees . . . . .                           | 488        |
| 11.8      | Lab: Survival Analysis . . . . .                   | 489        |
| 11.8.1    | Brain Cancer Data . . . . .                        | 489        |
| 11.8.2    | Publication Data . . . . .                         | 493        |
| 11.8.3    | Call Center Data . . . . .                         | 494        |
| 11.9      | Exercises . . . . .                                | 498        |
| <b>12</b> | <b>Unsupervised Learning</b>                       | <b>503</b> |
| 12.1      | The Challenge of Unsupervised Learning . . . . .   | 503        |
| 12.2      | Principal Components Analysis . . . . .            | 504        |
| 12.2.1    | What Are Principal Components? . . . . .           | 505        |
| 12.2.2    | Another Interpretation of Principal Components .   | 508        |
| 12.2.3    | The Proportion of Variance Explained . . . . .     | 510        |
| 12.2.4    | More on PCA . . . . .                              | 512        |
| 12.2.5    | Other Uses for Principal Components . . . . .      | 515        |
| 12.3      | Missing Values and Matrix Completion . . . . .     | 515        |
| 12.4      | Clustering Methods . . . . .                       | 520        |
| 12.4.1    | $K$ -Means Clustering . . . . .                    | 521        |
| 12.4.2    | Hierarchical Clustering . . . . .                  | 525        |
| 12.4.3    | Practical Issues in Clustering . . . . .           | 532        |
| 12.5      | Lab: Unsupervised Learning . . . . .               | 535        |
| 12.5.1    | Principal Components Analysis . . . . .            | 535        |
| 12.5.2    | Matrix Completion . . . . .                        | 539        |
| 12.5.3    | Clustering . . . . .                               | 542        |
| 12.5.4    | NCI60 Data Example . . . . .                       | 546        |
| 12.6      | Exercises . . . . .                                | 552        |
| <b>13</b> | <b>Multiple Testing</b>                            | <b>557</b> |
| 13.1      | A Quick Review of Hypothesis Testing . . . . .     | 558        |
| 13.1.1    | Testing a Hypothesis . . . . .                     | 558        |
| 13.1.2    | Type I and Type II Errors . . . . .                | 562        |
| 13.2      | The Challenge of Multiple Testing . . . . .        | 563        |
| 13.3      | The Family-Wise Error Rate . . . . .               | 565        |
| 13.3.1    | What is the Family-Wise Error Rate? . . . . .      | 565        |
| 13.3.2    | Approaches to Control the Family-Wise Error Rate   | 567        |
| 13.3.3    | Trade-Off Between the FWER and Power . . . . .     | 572        |
| 13.4      | The False Discovery Rate . . . . .                 | 573        |
| 13.4.1    | Intuition for the False Discovery Rate . . . . .   | 573        |
| 13.4.2    | The Benjamini–Hochberg Procedure . . . . .         | 575        |

|        |                                                                           |            |
|--------|---------------------------------------------------------------------------|------------|
| 13.5   | A Re-Sampling Approach to $p$ -Values and False Discovery Rates . . . . . | 577        |
| 13.5.1 | A Re-Sampling Approach to the $p$ -Value . . . . .                        | 578        |
| 13.5.2 | A Re-Sampling Approach to the False Discovery Rate                        | 579        |
| 13.5.3 | When Are Re-Sampling Approaches Useful? . . . .                           | 581        |
| 13.6   | Lab: Multiple Testing . . . . .                                           | 583        |
| 13.6.1 | Review of Hypothesis Tests . . . . .                                      | 583        |
| 13.6.2 | Family-Wise Error Rate . . . . .                                          | 585        |
| 13.6.3 | False Discovery Rate . . . . .                                            | 588        |
| 13.6.4 | A Re-Sampling Approach . . . . .                                          | 590        |
| 13.7   | Exercises . . . . .                                                       | 593        |
|        | <b>Index</b>                                                              | <b>597</b> |



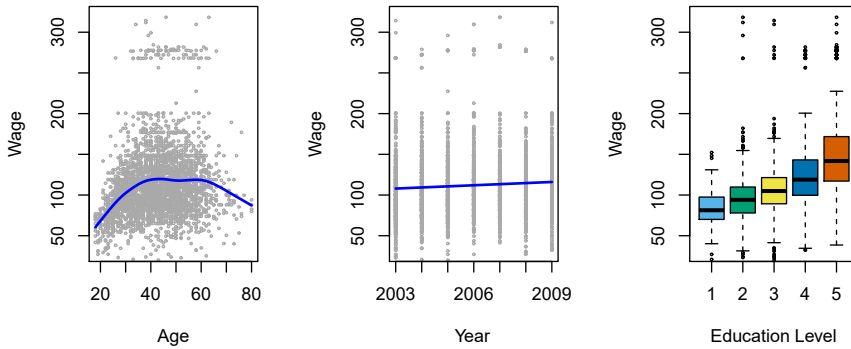
# Introduction

## An Overview of Statistical Learning

*Statistical learning* refers to a vast set of tools for *understanding data*. These tools can be classified as *supervised* or *unsupervised*. Broadly speaking, supervised statistical learning involves building a statistical model for predicting, or estimating, an *output* based on one or more *inputs*. Problems of this nature occur in fields as diverse as business, medicine, astrophysics, and public policy. With unsupervised statistical learning, there are inputs but no supervising output; nevertheless we can learn relationships and structure from such data. To provide an illustration of some applications of statistical learning, we briefly discuss three real-world data sets that are considered in this book.

### *Wage Data*

In this application (which we refer to as the *Wage* data set throughout this book), we examine a number of factors that relate to wages for a group of men from the Atlantic region of the United States. In particular, we wish to understand the association between an employee's *age* and *education*, as well as the calendar *year*, on his *wage*. Consider, for example, the left-hand panel of Figure 1.1, which displays *wage* versus *age* for each of the individuals in the data set. There is evidence that *wage* increases with *age* but then decreases again after approximately age 60. The blue line, which provides an estimate of the average *wage* for a given *age*, makes this trend clearer. Given an employee's *age*, we can use this curve to *predict* his *wage*. However, it is also clear from Figure 1.1 that there is a significant amount of variability associated with this average value, and so *age* alone is unlikely to provide an accurate prediction of a particular man's *wage*.

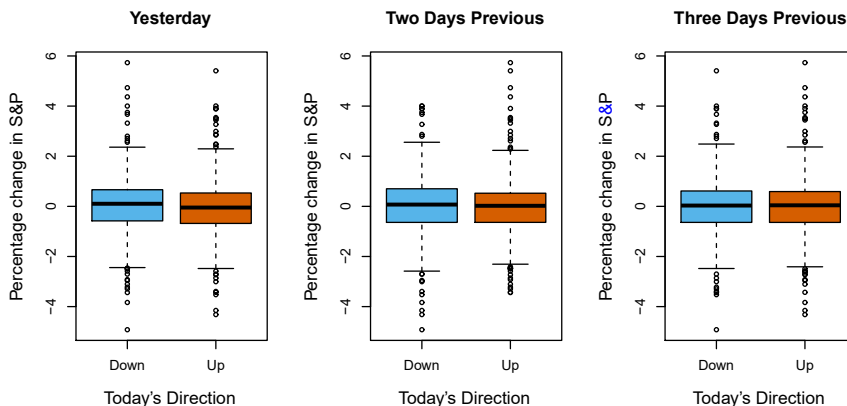


**FIGURE 1.1.** Wage data, which contains income survey information for men from the central Atlantic region of the United States. Left: wage as a function of age. On average, wage increases with age until about 60 years of age, at which point it begins to decline. Center: wage as a function of year. There is a slow but steady increase of approximately \$10,000 in the average wage between 2003 and 2009. Right: Boxplots displaying wage as a function of education, with 1 indicating the lowest level (no high school diploma) and 5 the highest level (an advanced graduate degree). On average, wage increases with the level of education.

We also have information regarding each employee’s education level and the year in which the wage was earned. The center and right-hand panels of Figure 1.1, which display wage as a function of both year and education, indicate that both of these factors are associated with wage. Wages increase by approximately \$10,000, in a roughly linear (or straight-line) fashion, between 2003 and 2009, though this rise is very slight relative to the variability in the data. Wages are also typically greater for individuals with higher education levels: men with the lowest education level (1) tend to have substantially lower wages than those with the highest education level (5). Clearly, the most accurate prediction of a given man’s wage will be obtained by combining his age, his education, and the year. In Chapter 3, we discuss linear regression, which can be used to predict wage from this data set. Ideally, we should predict wage in a way that accounts for the non-linear relationship between wage and age. In Chapter 7, we discuss a class of approaches for addressing this problem.

### Stock Market Data

The Wage data involves predicting a *continuous* or *quantitative* output value. This is often referred to as a *regression* problem. However, in certain cases we may instead wish to predict a non-numerical value—that is, a *categorical* or *qualitative* output. For example, in Chapter 4 we examine a stock market data set that contains the daily movements in the Standard & Poor’s 500 (S&P) stock index over a 5-year period between 2001 and 2005. We refer to this as the Smarket data. The goal is to predict whether the index will *increase* or *decrease* on a given day, using the past 5 days’ percentage changes in the index. Here the statistical learning problem does not involve predicting a numerical value. Instead it involves predicting whether a given



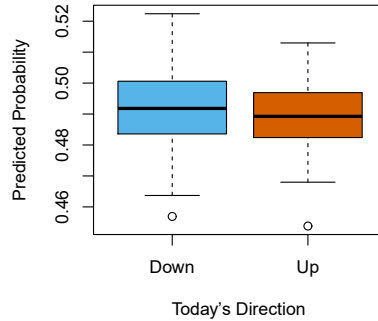
**FIGURE 1.2.** Left: Boxplots of the previous day's percentage change in the S&P index for the days for which the market increased or decreased, obtained from the **Smarket** data. Center and Right: Same as left panel, but the percentage changes for 2 and 3 days previous are shown.

day's stock market performance will fall into the **Up** bucket or the **Down** bucket. This is known as a *classification* problem. A model that could accurately predict the direction in which the market will move would be very useful!

The left-hand panel of Figure 1.2 displays two boxplots of the previous day's percentage changes in the stock index: one for the 648 days for which the market increased on the subsequent day, and one for the 602 days for which the market decreased. The two plots look almost identical, suggesting that there is no simple strategy for using yesterday's movement in the S&P to predict today's returns. The remaining panels, which display boxplots for the percentage changes 2 and 3 days previous to today, similarly indicate little association between past and present returns. Of course, this lack of pattern is to be expected: in the presence of strong correlations between successive days' returns, one could adopt a simple trading strategy to generate profits from the market. Nevertheless, in Chapter 4, we explore these data using several different statistical learning methods. Interestingly, there are hints of some weak trends in the data that suggest that, at least for this 5-year period, it is possible to correctly predict the direction of movement in the market approximately 60% of the time (Figure 1.3).

## Gene Expression Data

The previous two applications illustrate data sets with both input and output variables. However, another important class of problems involves situations in which we only observe input variables, with no corresponding output. For example, in a marketing setting, we might have demographic information for a number of current or potential customers. We may wish to understand which types of customers are similar to each other by grouping individuals according to their observed characteristics. This is known as a



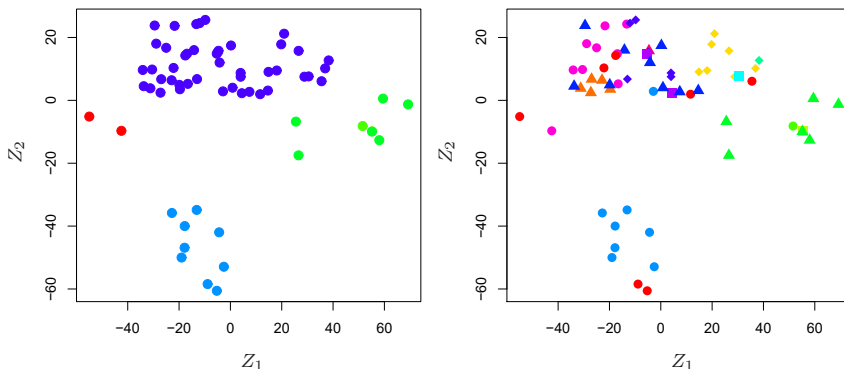
**FIGURE 1.3.** We fit a quadratic discriminant analysis model to the subset of the **Smarket** data corresponding to the 2001–2004 time period, and predicted the probability of a stock market decrease using the 2005 data. On average, the predicted probability of decrease is higher for the days in which the market does decrease. Based on these results, we are able to correctly predict the direction of movement in the market 60% of the time.

*clustering* problem. Unlike in the previous examples, here we are not trying to predict an output variable.

We devote Chapter 12 to a discussion of statistical learning methods for problems in which no natural output variable is available. We consider the **NCI60** data set, which consists of 6,830 gene expression measurements for each of 64 cancer cell lines. Instead of predicting a particular output variable, we are interested in determining whether there are groups, or clusters, among the cell lines based on their gene expression measurements. This is a difficult question to address, in part because there are thousands of gene expression measurements per cell line, making it hard to visualize the data.

The left-hand panel of Figure 1.4 addresses this problem by representing each of the 64 cell lines using just two numbers,  $Z_1$  and  $Z_2$ . These are the first two *principal components* of the data, which summarize the 6,830 expression measurements for each cell line down to two numbers or *dimensions*. While it is likely that this dimension reduction has resulted in some loss of information, it is now possible to visually examine the data for evidence of clustering. Deciding on the number of clusters is often a difficult problem. But the left-hand panel of Figure 1.4 suggests at least four groups of cell lines, which we have represented using separate colors.

In this particular data set, it turns out that the cell lines correspond to 14 different types of cancer. (However, this information was not used to create the left-hand panel of Figure 1.4.) The right-hand panel of Figure 1.4 is identical to the left-hand panel, except that the 14 cancer types are shown using distinct colored symbols. There is clear evidence that cell lines with the same cancer type tend to be located near each other in this two-dimensional representation. In addition, even though the cancer information was not used to produce the left-hand panel, the clustering obtained does bear some resemblance to some of the actual cancer types observed in the right-hand panel. This provides some independent verification of the accuracy of our clustering analysis.



**FIGURE 1.4.** Left: Representation of the NCI60 gene expression data set in a two-dimensional space,  $Z_1$  and  $Z_2$ . Each point corresponds to one of the 64 cell lines. There appear to be four groups of cell lines, which we have represented using different colors. Right: Same as left panel except that we have represented each of the 14 different types of cancer using a different colored symbol. Cell lines corresponding to the same cancer type tend to be nearby in the two-dimensional space.

## A Brief History of Statistical Learning

Though the term *statistical learning* is fairly new, many of the concepts that underlie the field were developed long ago. At the beginning of the nineteenth century, the method of *least squares* was developed, implementing the earliest form of what is now known as *linear regression*. The approach was first successfully applied to problems in astronomy. Linear regression is used for predicting quantitative values, such as an individual's salary. In order to predict qualitative values, such as whether a patient survives or dies, or whether the stock market increases or decreases, *linear discriminant analysis* was proposed in 1936. In the 1940s, various authors put forth an alternative approach, *logistic regression*. In the early 1970s, the term *generalized linear model* was developed to describe an entire class of statistical learning methods that include both linear and logistic regression as special cases.

By the end of the 1970s, many more techniques for learning from data were available. However, they were almost exclusively *linear* methods because fitting *non-linear* relationships was computationally difficult at the time. By the 1980s, computing technology had finally improved sufficiently that non-linear methods were no longer computationally prohibitive. In the mid 1980s, *classification and regression trees* were developed, followed shortly by *generalized additive models*. *Neural networks* gained popularity in the 1980s, and *support vector machines* arose in the 1990s.

Since that time, statistical learning has emerged as a new subfield in statistics, focused on supervised and unsupervised modeling and prediction. In recent years, progress in statistical learning has been marked by the increasing availability of powerful and relatively user-friendly software, such as the popular and freely available `Python` system. This has the potential to continue the transformation of the field from a set of techniques used and



developed by statisticians and computer scientists to an essential toolkit for a much broader community.

## This Book

*The Elements of Statistical Learning* (ESL) by Hastie, Tibshirani, and Friedman was first published in 2001. Since that time, it has become an important reference on the fundamentals of statistical machine learning. Its success derives from its comprehensive and detailed treatment of many important topics in statistical learning, as well as the fact that (relative to many upper-level statistics textbooks) it is accessible to a wide audience. However, the greatest factor behind the success of ESL has been its topical nature. At the time of its publication, interest in the field of statistical learning was starting to explode. ESL provided one of the first accessible and comprehensive introductions to the topic.

Since ESL was first published, the field of statistical learning has continued to flourish. The field's expansion has taken two forms. The most obvious growth has involved the development of new and improved statistical learning approaches aimed at answering a range of scientific questions across a number of fields. However, the field of statistical learning has also expanded its audience. In the 1990s, increases in computational power generated a surge of interest in the field from non-statisticians who were eager to use cutting-edge statistical tools to analyze their data. Unfortunately, the highly technical nature of these approaches meant that the user community remained primarily restricted to experts in statistics, computer science, and related fields with the training (and time) to understand and implement them.

In recent years, new and improved software packages have significantly eased the implementation burden for many statistical learning methods. At the same time, there has been growing recognition across a number of fields, from business to health care to genetics to the social sciences and beyond, that statistical learning is a powerful tool with important practical applications. As a result, the field has moved from one of primarily academic interest to a mainstream discipline, with an enormous potential audience. This trend will surely continue with the increasing availability of enormous quantities of data and the software to analyze it.

The purpose of *An Introduction to Statistical Learning* (ISL) is to facilitate the transition of statistical learning from an academic to a mainstream field. ISL is not intended to replace ESL, which is a far more comprehensive text both in terms of the number of approaches considered and the depth to which they are explored. We consider ESL to be an important companion for professionals (with graduate degrees in statistics, machine learning, or related fields) who need to understand the technical details behind statistical learning approaches. However, the community of users of statistical learning techniques has expanded to include individuals with a wider range of interests and backgrounds. Therefore, there is a place for a less technical and more accessible version of ESL.

In teaching these topics over the years, we have discovered that they are of interest to master's and PhD students in fields as disparate as business administration, biology, and computer science, as well as to quantitatively-oriented upper-division undergraduates. It is important for this diverse group to be able to understand the models, intuitions, and strengths and weaknesses of the various approaches. But for this audience, many of the technical details behind statistical learning methods, such as optimization algorithms and theoretical properties, are not of primary interest. We believe that these students do not need a deep understanding of these aspects in order to become informed users of the various methodologies, and in order to contribute to their chosen fields through the use of statistical learning tools.

ISL is based on the following four premises.

1. *Many statistical learning methods are relevant and useful in a wide range of academic and non-academic disciplines, beyond just the statistical sciences.* We believe that many contemporary statistical learning procedures should, and will, become as widely available and used as is currently the case for classical methods such as linear regression. As a result, rather than attempting to consider every possible approach (an impossible task), we have concentrated on presenting the methods that we believe are most widely applicable.
2. *Statistical learning should not be viewed as a series of black boxes.* No single approach will perform well in all possible applications. Without understanding all of the cogs inside the box, or the interaction between those cogs, it is impossible to select the best box. Hence, we have attempted to carefully describe the model, intuition, assumptions, and trade-offs behind each of the methods that we consider.
3. *While it is important to know what job is performed by each cog, it is not necessary to have the skills to construct the machine inside the box!* Thus, we have minimized discussion of technical details related to fitting procedures and theoretical properties. We assume that the reader is comfortable with basic mathematical concepts, but we do not assume a graduate degree in the mathematical sciences. For instance, we have almost completely avoided the use of matrix algebra, and it is possible to understand the entire book without a detailed knowledge of matrices and vectors.
4. *We presume that the reader is interested in applying statistical learning methods to real-world problems.* In order to facilitate this, as well as to motivate the techniques discussed, we have devoted a section within each chapter to computer labs. In each lab, we walk the reader through a realistic application of the methods considered in that chapter. When we have taught this material in our courses, we have allocated roughly one-third of classroom time to working through the labs, and we have found them to be extremely useful. Many of the less computationally-oriented students who were initially intimidated by the labs got the hang of things over the course of the quarter or semester. This book originally appeared (2013, second edition 2021)

with computer labs written in the **R** language. Since then, there has been increasing demand for **Python** implementations of the important techniques in statistical learning. Consequently, this version has labs in **Python**. There are a rapidly growing number of **Python** packages available, and by examination of the imports at the beginning of each lab, readers will see that we have carefully selected and used the most appropriate. We have also supplied some additional code and functionality in our package **ISLP**. However, the labs in **ISL** are self-contained, and can be skipped if the reader wishes to use a different software package or does not wish to apply the methods discussed to real-world problems.

## Who Should Read This Book?

This book is intended for anyone who is interested in using modern statistical methods for modeling and prediction from data. This group includes scientists, engineers, data analysts, data scientists, and quants, but also less technical individuals with degrees in non-quantitative fields such as the social sciences or business. We expect that the reader will have had at least one elementary course in statistics. Background in linear regression is also useful, though not required, since we review the key concepts behind linear regression in Chapter 3. The mathematical level of this book is modest, and a detailed knowledge of matrix operations is not required. This book provides an introduction to **Python**. Previous exposure to a programming language, such as **MATLAB** or **R**, is useful but not required.

The first edition of this textbook has been used to teach master's and PhD students in business, economics, computer science, biology, earth sciences, psychology, and many other areas of the physical and social sciences. It has also been used to teach advanced undergraduates who have already taken a course on linear regression. In the context of a more mathematically rigorous course in which **ESL** serves as the primary textbook, **ISL** could be used as a supplementary text for teaching computational aspects of the various approaches.

## Notation and Simple Matrix Algebra

Choosing notation for a textbook is always a difficult task. For the most part we adopt the same notational conventions as **ESL**.

We will use  $n$  to represent the number of distinct data points, or observations, in our sample. We will let  $p$  denote the number of variables that are available for use in making predictions. For example, the **Wage** data set consists of 11 variables for 3,000 people, so we have  $n = 3,000$  observations and  $p = 11$  variables (such as **year**, **age**, **race**, and more). Note that throughout this book, we indicate variable names using colored font: **Variable Name**.

In some examples,  $p$  might be quite large, such as on the order of thousands or even millions; this situation arises quite often, for example, in the analysis of modern biological data or web-based advertising data.

In general, we will let  $x_{ij}$  represent the value of the  $j$ th variable for the  $i$ th observation, where  $i = 1, 2, \dots, n$  and  $j = 1, 2, \dots, p$ . Throughout this book,  $i$  will be used to index the samples or observations (from 1 to  $n$ ) and  $j$  will be used to index the variables (from 1 to  $p$ ). We let  $\mathbf{X}$  denote an  $n \times p$  matrix whose  $(i, j)$ th element is  $x_{ij}$ . That is,

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1p} \\ x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \dots & x_{np} \end{pmatrix}.$$

For readers who are unfamiliar with matrices, it is useful to visualize  $\mathbf{X}$  as a spreadsheet of numbers with  $n$  rows and  $p$  columns.

At times we will be interested in the rows of  $\mathbf{X}$ , which we write as  $x_1, x_2, \dots, x_n$ . Here  $x_i$  is a vector of length  $p$ , containing the  $p$  variable measurements for the  $i$ th observation. That is,

$$x_i = \begin{pmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{ip} \end{pmatrix}. \quad (1.1)$$

(Vectors are by default represented as columns.) For example, for the **Wage** data,  $x_i$  is a vector of length 11, consisting of **year**, **age**, **race**, and other values for the  $i$ th individual. At other times we will instead be interested in the columns of  $\mathbf{X}$ , which we write as  $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_p$ . Each is a vector of length  $n$ . That is,

$$\mathbf{x}_j = \begin{pmatrix} x_{1j} \\ x_{2j} \\ \vdots \\ x_{nj} \end{pmatrix}.$$

For example, for the **Wage** data,  $\mathbf{x}_1$  contains the  $n = 3,000$  values for **year**.

Using this notation, the matrix  $\mathbf{X}$  can be written as

$$\mathbf{X} = (\mathbf{x}_1 \quad \mathbf{x}_2 \quad \dots \quad \mathbf{x}_p),$$

or

$$\mathbf{X} = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_n^T \end{pmatrix}.$$

The  $^T$  notation denotes the *transpose* of a matrix or vector. So, for example,

$$\mathbf{X}^T = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1p} \\ x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \dots & x_{np} \end{pmatrix},$$

while

$$x_i^T = (x_{i1} \quad x_{i2} \quad \cdots \quad x_{ip}).$$

We use  $y_i$  to denote the  $i$ th observation of the variable on which we wish to make predictions, such as **wage**. Hence, we write the set of all  $n$  observations in vector form as

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}.$$

Then our observed data consists of  $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ , where each  $x_i$  is a vector of length  $p$ . (If  $p = 1$ , then  $x_i$  is simply a scalar.)

In this text, a vector of length  $n$  will always be denoted in *lower case bold*; e.g.

$$\mathbf{a} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix}.$$

However, vectors that are not of length  $n$  (such as feature vectors of length  $p$ , as in (1.1)) will be denoted in *lower case normal font*, e.g.  $a$ . Scalars will also be denoted in *lower case normal font*, e.g.  $a$ . In the rare cases in which these two uses for lower case normal font lead to ambiguity, we will clarify which use is intended. Matrices will be denoted using *bold capitals*, such as  $\mathbf{A}$ . Random variables will be denoted using *capital normal font*, e.g.  $A$ , regardless of their dimensions.

Occasionally we will want to indicate the dimension of a particular object. To indicate that an object is a scalar, we will use the notation  $a \in \mathbb{R}$ . To indicate that it is a vector of length  $k$ , we will use  $a \in \mathbb{R}^k$  (or  $\mathbf{a} \in \mathbb{R}^n$  if it is of length  $n$ ). We will indicate that an object is an  $r \times s$  matrix using  $\mathbf{A} \in \mathbb{R}^{r \times s}$ .

We have avoided using matrix algebra whenever possible. However, in a few instances it becomes too cumbersome to avoid it entirely. In these rare instances it is important to understand the concept of multiplying two matrices. Suppose that  $\mathbf{A} \in \mathbb{R}^{r \times d}$  and  $\mathbf{B} \in \mathbb{R}^{d \times s}$ . Then the product of  $\mathbf{A}$  and  $\mathbf{B}$  is denoted  $\mathbf{AB}$ . The  $(i, j)$ th element of  $\mathbf{AB}$  is computed by multiplying each element of the  $i$ th row of  $\mathbf{A}$  by the corresponding element of the  $j$ th column of  $\mathbf{B}$ . That is,  $(\mathbf{AB})_{ij} = \sum_{k=1}^d a_{ik}b_{kj}$ . As an example, consider

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad \text{and} \quad \mathbf{B} = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}.$$

Then

$$\mathbf{AB} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 1 \times 5 + 2 \times 7 & 1 \times 6 + 2 \times 8 \\ 3 \times 5 + 4 \times 7 & 3 \times 6 + 4 \times 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}.$$

Note that this operation produces an  $r \times s$  matrix. It is only possible to compute  $\mathbf{AB}$  if the number of columns of  $\mathbf{A}$  is the same as the number of rows of  $\mathbf{B}$ .

## Organization of This Book

Chapter 2 introduces the basic terminology and concepts behind statistical learning. This chapter also presents the *K-nearest neighbor* classifier, a very simple method that works surprisingly well on many problems. Chapters 3 and 4 cover classical linear methods for regression and classification. In particular, Chapter 3 reviews *linear regression*, the fundamental starting point for all regression methods. In Chapter 4 we discuss two of the most important classical classification methods, *logistic regression* and *linear discriminant analysis*.

A central problem in all statistical learning situations involves choosing the best method for a given application. Hence, in Chapter 5 we introduce *cross-validation* and the *bootstrap*, which can be used to estimate the accuracy of a number of different methods in order to choose the best one.

Much of the recent research in statistical learning has concentrated on non-linear methods. However, linear methods often have advantages over their non-linear competitors in terms of interpretability and sometimes also accuracy. Hence, in Chapter 6 we consider a host of linear methods, both classical and more modern, which offer potential improvements over standard linear regression. These include *stepwise selection*, *ridge regression*, *principal components regression*, and the *lasso*.

The remaining chapters move into the world of non-linear statistical learning. We first introduce in Chapter 7 a number of non-linear methods that work well for problems with a single input variable. We then show how these methods can be used to fit non-linear *additive* models for which there is more than one input. In Chapter 8, we investigate *tree-based* methods, including *bagging*, *boosting*, and *random forests*. *Support vector machines*, a set of approaches for performing both linear and non-linear classification, are discussed in Chapter 9. We cover *deep learning*, an approach for non-linear regression and classification that has received a lot of attention in recent years, in Chapter 10. Chapter 11 explores *survival analysis*, a regression approach that is specialized to the setting in which the output variable is *censored*, i.e. not fully observed.

In Chapter 12, we consider the *unsupervised* setting in which we have input variables but no output variable. In particular, we present *principal components analysis*, *K-means clustering*, and *hierarchical clustering*. Finally, in Chapter 13 we cover the very important topic of multiple hypothesis testing.

At the end of each chapter, we present one or more **Python** lab sections in which we systematically work through applications of the various methods discussed in that chapter. These labs demonstrate the strengths and weaknesses of the various approaches, and also provide a useful reference for the syntax required to implement the various methods. The reader may choose to work through the labs at their own pace, or the labs may be the focus of group sessions as part of a classroom environment. Within each **Python** lab, we present the results that we obtained when we performed the lab at the time of writing this book. However, new versions of **Python** are continuously released, and over time, the packages called in the labs will be updated. Therefore, in the future, it is possible that the results shown in

| Name               | Description                                                       |
|--------------------|-------------------------------------------------------------------|
| <b>Auto</b>        | Gas mileage, horsepower, and other information for cars.          |
| <b>Bikeshare</b>   | Hourly usage of a bike sharing program in Washington, DC.         |
| <b>Boston</b>      | Housing values and other information about Boston census tracts.  |
| <b>BrainCancer</b> | Survival times for patients diagnosed with brain cancer.          |
| <b>Caravan</b>     | Information about individuals offered caravan insurance.          |
| <b>Carseats</b>    | Information about car seat sales in 400 stores.                   |
| <b>College</b>     | Demographic characteristics, tuition, and more for USA colleges.  |
| <b>Credit</b>      | Information about credit card debt for 400 customers.             |
| <b>Default</b>     | Customer default records for a credit card company.               |
| <b>Fund</b>        | Returns of 2,000 hedge fund managers over 50 months.              |
| <b>Hitters</b>     | Records and salaries for baseball players.                        |
| <b>Khan</b>        | Gene expression measurements for four cancer types.               |
| <b>NCI60</b>       | Gene expression measurements for 64 cancer cell lines.            |
| <b>NYSE</b>        | Returns, volatility, and volume for the New York Stock Exchange.  |
| <b>OJ</b>          | Sales information for Citrus Hill and Minute Maid orange juice.   |
| <b>Portfolio</b>   | Past values of financial assets, for use in portfolio allocation. |
| <b>Publication</b> | Time to publication for 244 clinical trials.                      |
| <b>Smarket</b>     | Daily percentage returns for S&P 500 over a 5-year period.        |
| <b>USArrests</b>   | Crime statistics per 100,000 residents in 50 states of USA.       |
| <b>Wage</b>        | Income survey data for men in central Atlantic region of USA.     |
| <b>Weekly</b>      | 1,089 weekly stock market returns for 21 years.                   |

**TABLE 1.1.** *A list of data sets needed to perform the labs and exercises in this textbook. All data sets are available in the **ISLP** package, with the exception of **USArrests**, which is part of the base **R** distribution, but accessible from **Python**.*

the lab sections may no longer correspond precisely to the results obtained by the reader who performs the labs. As necessary, we will post updates to the labs on the book website.

We use the  symbol to denote sections or exercises that contain more challenging concepts. These can be easily skipped by readers who do not wish to delve as deeply into the material, or who lack the mathematical background.

## Data Sets Used in Labs and Exercises

In this textbook, we illustrate statistical learning methods using applications from marketing, finance, biology, and other areas. The **ISLP** package contains a number of data sets that are required in order to perform the labs and exercises associated with this book. One other data set is part of the base **R** distribution (the **USArrests** data), and we show how to access it from **Python** in Section 12.5.1. Table 1.1 contains a summary of the data sets required to perform the labs and exercises. A couple of these data sets are also available as text files on the book website, for use in Chapter 2.

## Book Website

The website for this book is located at

[www.statlearning.com](http://www.statlearning.com)

It contains a number of resources, including the **Python** package associated with this book, and some additional data sets.

## Acknowledgements

A few of the plots in this book were taken from ESL: Figures 6.7, 8.3, and 12.14. All other plots were produced for the **R** version of ISL, except for Figure 13.10 which differs because of the **Python** software supporting the plot.





## 2.1 What Is Statistical Learning?

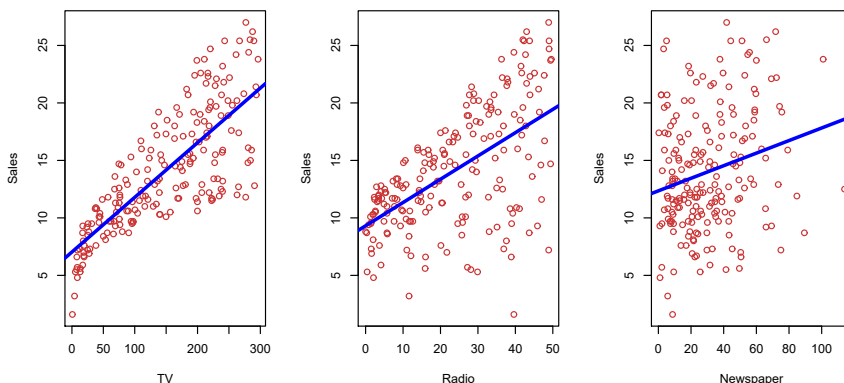
In order to motivate our study of statistical learning, we begin with a simple example. Suppose that we are statistical consultants hired by a client to investigate the association between advertising and sales of a particular product. The **Advertising** data set consists of the **sales** of that product in 200 different markets, along with advertising budgets for the product in each of those markets for three different media: **TV**, **radio**, and **newspaper**. The data are displayed in Figure 2.1. It is not possible for our client to directly increase sales of the product. On the other hand, they can control the advertising expenditure in each of the three media. Therefore, if we determine that there is an association between advertising and sales, then we can instruct our client to adjust advertising budgets, thereby indirectly increasing sales. In other words, our goal is to develop an accurate model that can be used to predict sales on the basis of the three media budgets.

In this setting, the advertising budgets are *input variables* while **sales** is an *output variable*. The input variables are typically denoted using the symbol  $X$ , with a subscript to distinguish them. So  $X_1$  might be the **TV** budget,  $X_2$  the **radio** budget, and  $X_3$  the **newspaper** budget. The inputs go by different names, such as *predictors*, *independent variables*, *features*, or sometimes just *variables*. The output variable—in this case, **sales**—is often called the *response* or *dependent variable*, and is typically denoted using the symbol  $Y$ . Throughout this book, we will use all of these terms interchangeably.

More generally, suppose that we observe a quantitative response  $Y$  and  $p$  different predictors,  $X_1, X_2, \dots, X_p$ . We assume that there is some relationship between  $Y$  and  $X = (X_1, X_2, \dots, X_p)$ , which can be written in the very general form

$$Y = f(X) + \epsilon. \quad (2.1)$$

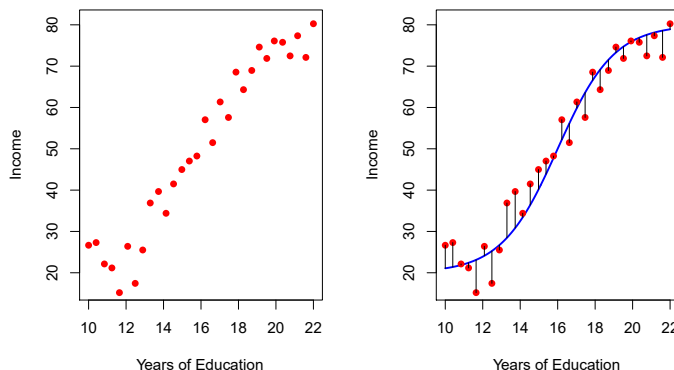
input  
variable  
output  
variable  
  
predictor  
independent  
variable  
feature  
variable  
response  
dependent  
variable



**FIGURE 2.1.** The *Advertising* data set. The plot displays *sales*, in thousands of units, as a function of *TV*, *radio*, and *newspaper* budgets, in thousands of dollars, for 200 different markets. In each plot we show the simple least squares fit of *sales* to that variable, as described in Chapter 3. In other words, each blue line represents a simple model that can be used to predict *sales* using *TV*, *radio*, and *newspaper*, respectively.

Here  $f$  is some fixed but unknown function of  $X_1, \dots, X_p$ , and  $\epsilon$  is a random error term, which is independent of  $X$  and has mean zero. In this formulation,  $f$  represents the *systematic* information that  $X$  provides about  $Y$ .

error term  
systematic



**FIGURE 2.2.** The *Income* data set. Left: The red dots are the observed values of *income* (in thousands of dollars) and *years of education* for 30 individuals. Right: The blue curve represents the true underlying relationship between *income* and *years of education*, which is generally unknown (but is known in this case because the data were simulated). The black lines represent the error associated with each observation. Note that some errors are positive (if an observation lies above the blue curve) and some are negative (if an observation lies below the curve). Overall, these errors have approximately mean zero.

As another example, consider the left-hand panel of Figure 2.2, a plot of *income* versus *years of education* for 30 individuals in the *Income* data set. The plot suggests that one might be able to predict *income* using *years of education*. However, the function  $f$  that connects the input variable to the

output variable is in general unknown. In this situation one must estimate  $f$  based on the observed points. Since **Income** is a simulated data set,  $f$  is known and is shown by the blue curve in the right-hand panel of Figure 2.2. The vertical lines represent the error terms  $\epsilon$ . We note that some of the 30 observations lie above the blue curve and some lie below it; overall, the errors have approximately mean zero.

In general, the function  $f$  may involve more than one input variable. In Figure 2.3 we plot **income** as a function of **years of education** and **seniority**. Here  $f$  is a two-dimensional surface that must be estimated based on the observed data.

In essence, statistical learning refers to a set of approaches for estimating  $f$ . In this chapter we outline some of the key theoretical concepts that arise in estimating  $f$ , as well as tools for evaluating the estimates obtained.

### 2.1.1 Why Estimate $f$ ?

There are two main reasons that we may wish to estimate  $f$ : *prediction* and *inference*. We discuss each in turn.

#### Prediction

In many situations, a set of inputs  $X$  are readily available, but the output  $Y$  cannot be easily obtained. In this setting, since the error term averages to zero, we can predict  $Y$  using

$$\hat{Y} = \hat{f}(X), \quad (2.2)$$

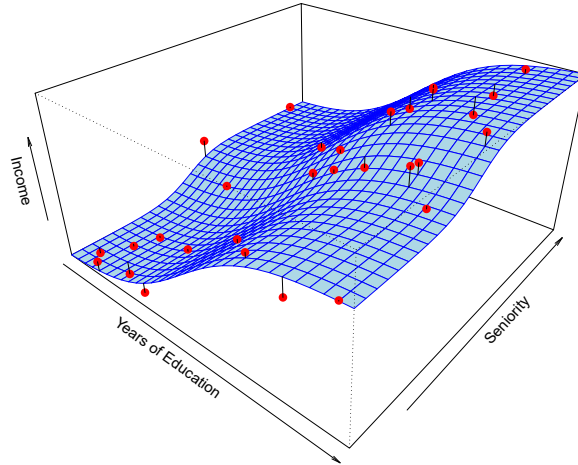
where  $\hat{f}$  represents our estimate for  $f$ , and  $\hat{Y}$  represents the resulting prediction for  $Y$ . In this setting,  $\hat{f}$  is often treated as a *black box*, in the sense that one is not typically concerned with the exact form of  $\hat{f}$ , provided that it yields accurate predictions for  $Y$ .

As an example, suppose that  $X_1, \dots, X_p$  are characteristics of a patient's blood sample that can be easily measured in a lab, and  $Y$  is a variable encoding the patient's risk for a severe adverse reaction to a particular drug. It is natural to seek to predict  $Y$  using  $X$ , since we can then avoid giving the drug in question to patients who are at high risk of an adverse reaction—that is, patients for whom the estimate of  $Y$  is high.

The accuracy of  $\hat{Y}$  as a prediction for  $Y$  depends on two quantities, which we will call the *reducible error* and the *irreducible error*. In general,  $\hat{f}$  will not be a perfect estimate for  $f$ , and this inaccuracy will introduce some error. This error is *reducible* because we can potentially improve the accuracy of  $\hat{f}$  by using the most appropriate statistical learning technique to estimate  $f$ . However, even if it were possible to form a perfect estimate for  $f$ , so that our estimated response took the form  $\hat{Y} = f(X)$ , our prediction would still have some error in it! This is because  $Y$  is also a function of  $\epsilon$ , which, by definition, cannot be predicted using  $X$ . Therefore, variability associated with  $\epsilon$  also affects the accuracy of our predictions. This is known as the *irreducible error*, because no matter how well we estimate  $f$ , we cannot reduce the error introduced by  $\epsilon$ .

Why is the irreducible error larger than zero? The quantity  $\epsilon$  may contain unmeasured variables that are useful in predicting  $Y$ : since we don't

reducible  
error  
irreducible  
error



**FIGURE 2.3.** The plot displays **income** as a function of **years of education** and **seniority** in the **Income** data set. The blue surface represents the true underlying relationship between **income** and **years of education** and **seniority**, which is known since the data are simulated. The red dots indicate the observed values of these quantities for 30 individuals.

measure them,  $f$  cannot use them for its prediction. The quantity  $\epsilon$  may also contain unmeasurable variation. For example, the risk of an adverse reaction might vary for a given patient on a given day, depending on manufacturing variation in the drug itself or the patient's general feeling of well-being on that day.

Consider a given estimate  $\hat{f}$  and a set of predictors  $X$ , which yields the prediction  $\hat{Y} = \hat{f}(X)$ . Assume for a moment that both  $\hat{f}$  and  $X$  are fixed, so that the only variability comes from  $\epsilon$ . Then, it is easy to show that

$$\begin{aligned} E(Y - \hat{Y})^2 &= E[f(X) + \epsilon - \hat{f}(X)]^2 \\ &= \underbrace{[f(X) - \hat{f}(X)]^2}_{\text{Reducible}} + \underbrace{\text{Var}(\epsilon)}_{\text{Irreducible}}, \end{aligned} \quad (2.3)$$

where  $E(Y - \hat{Y})^2$  represents the average, or *expected value*, of the squared difference between the predicted and actual value of  $Y$ , and  $\text{Var}(\epsilon)$  represents the *variance* associated with the error term  $\epsilon$ .

The focus of this book is on techniques for estimating  $f$  with the aim of minimizing the reducible error. It is important to keep in mind that the irreducible error will always provide an upper bound on the accuracy of our prediction for  $Y$ . This bound is almost always unknown in practice.

## Inference

We are often interested in understanding the association between  $Y$  and  $X_1, \dots, X_p$ . In this situation we wish to estimate  $f$ , but our goal is not necessarily to make predictions for  $Y$ . Now  $\hat{f}$  cannot be treated as a black box, because we need to know its exact form. In this setting, one may be interested in answering the following questions:

- *Which predictors are associated with the response?* It is often the case that only a small fraction of the available predictors are substantially associated with  $Y$ . Identifying the few *important* predictors among a large set of possible variables can be extremely useful, depending on the application.
- *What is the relationship between the response and each predictor?* Some predictors may have a positive relationship with  $Y$ , in the sense that larger values of the predictor are associated with larger values of  $Y$ . Other predictors may have the opposite relationship. Depending on the complexity of  $f$ , the relationship between the response and a given predictor may also depend on the values of the other predictors.
- *Can the relationship between  $Y$  and each predictor be adequately summarized using a linear equation, or is the relationship more complicated?* Historically, most methods for estimating  $f$  have taken a linear form. In some situations, such an assumption is reasonable or even desirable. But often the true relationship is more complicated, in which case a linear model may not provide an accurate representation of the relationship between the input and output variables.

In this book, we will see a number of examples that fall into the prediction setting, the inference setting, or a combination of the two.

For instance, consider a company that is interested in conducting a direct-marketing campaign. The goal is to identify individuals who are likely to respond positively to a mailing, based on observations of demographic variables measured on each individual. In this case, the demographic variables serve as predictors, and response to the marketing campaign (either positive or negative) serves as the outcome. The company is not interested in obtaining a deep understanding of the relationships between each individual predictor and the response; instead, the company simply wants to accurately predict the response using the predictors. This is an example of modeling for prediction.

In contrast, consider the **Advertising** data illustrated in Figure 2.1. One may be interested in answering questions such as:

- *Which media are associated with sales?*
- *Which media generate the biggest boost in sales?* or
- *How large of an increase in sales is associated with a given increase in TV advertising?*

This situation falls into the inference paradigm. Another example involves modeling the brand of a product that a customer might purchase based on variables such as price, store location, discount levels, competition price, and so forth. In this situation one might really be most interested in the association between each variable and the probability of purchase. For instance, *to what extent is the product's price associated with sales?* This is an example of modeling for inference.

Finally, some modeling could be conducted both for prediction and inference. For example, in a real estate setting, one may seek to relate values

of homes to inputs such as crime rate, zoning, distance from a river, air quality, schools, income level of community, size of houses, and so forth. In this case one might be interested in the association between each individual input variable and housing price—for instance, *how much extra will a house be worth if it has a view of the river?* This is an inference problem. Alternatively, one may simply be interested in predicting the value of a home given its characteristics: *is this house under- or over-valued?* This is a prediction problem.

Depending on whether our ultimate goal is prediction, inference, or a combination of the two, different methods for estimating  $f$  may be appropriate. For example, *linear models* allow for relatively simple and interpretable inference, but may not yield as accurate predictions as some other approaches. In contrast, some of the highly non-linear approaches that we discuss in the later chapters of this book can potentially provide quite accurate predictions for  $Y$ , but this comes at the expense of a less interpretable model for which inference is more challenging.

linear model

### 2.1.2 How Do We Estimate $f$ ?

Throughout this book, we explore many linear and non-linear approaches for estimating  $f$ . However, these methods generally share certain characteristics. We provide an overview of these shared characteristics in this section. We will always assume that we have observed a set of  $n$  different data points. For example in Figure 2.2 we observed  $n = 30$  data points. These observations are called the *training data* because we will use these observations to train, or teach, our method how to estimate  $f$ . Let  $x_{ij}$  represent the value of the  $j$ th predictor, or input, for observation  $i$ , where  $i = 1, 2, \dots, n$  and  $j = 1, 2, \dots, p$ . Correspondingly, let  $y_i$  represent the response variable for the  $i$ th observation. Then our training data consist of  $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$  where  $x_i = (x_{i1}, x_{i2}, \dots, x_{ip})^T$ .

training data

Our goal is to apply a statistical learning method to the training data in order to estimate the unknown function  $f$ . In other words, we want to find a function  $\hat{f}$  such that  $Y \approx \hat{f}(X)$  for any observation  $(X, Y)$ . Broadly speaking, most statistical learning methods for this task can be characterized as either *parametric* or *non-parametric*. We now briefly discuss these two types of approaches.

parametric  
non-  
parametric

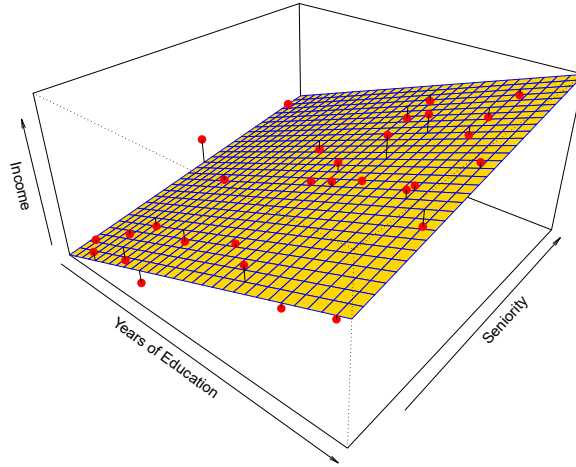
#### Parametric Methods

Parametric methods involve a two-step model-based approach.

1. First, we make an assumption about the functional form, or shape, of  $f$ . For example, one very simple assumption is that  $f$  is linear in  $X$ :

$$f(X) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p. \quad (2.4)$$

This is a *linear model*, which will be discussed extensively in Chapter 3. Once we have assumed that  $f$  is linear, the problem of estimating  $f$  is greatly simplified. Instead of having to estimate an entirely arbitrary  $p$ -dimensional function  $f(X)$ , one only needs to estimate the  $p + 1$  coefficients  $\beta_0, \beta_1, \dots, \beta_p$ .



**FIGURE 2.4.** A linear model fit by least squares to the **Income** data from Figure 2.3. The observations are shown in red, and the yellow plane indicates the least squares fit to the data.

- After a model has been selected, we need a procedure that uses the training data to *fit* or *train* the model. In the case of the linear model (2.4), we need to estimate the parameters  $\beta_0, \beta_1, \dots, \beta_p$ . That is, we want to find values of these parameters such that

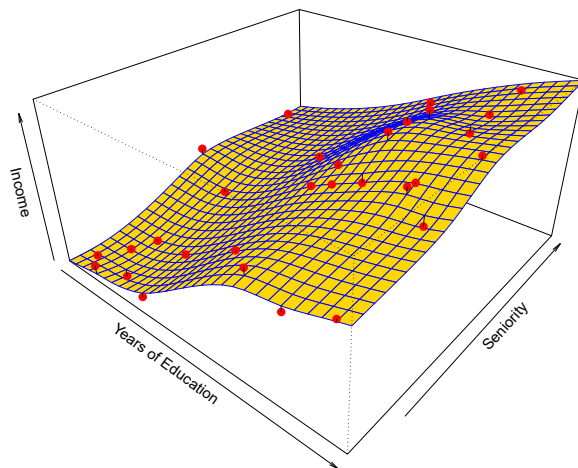
$$Y \approx \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p.$$

The most common approach to fitting the model (2.4) is referred to as (*ordinary*) *least squares*, which we discuss in Chapter 3. However, least squares is one of many possible ways to fit the linear model. In Chapter 6, we discuss other approaches for estimating the parameters in (2.4).

The model-based approach just described is referred to as *parametric*; it reduces the problem of estimating  $f$  down to one of estimating a set of parameters. Assuming a parametric form for  $f$  simplifies the problem of estimating  $f$  because it is generally much easier to estimate a set of parameters, such as  $\beta_0, \beta_1, \dots, \beta_p$  in the linear model (2.4), than it is to fit an entirely arbitrary function  $f$ . The potential disadvantage of a parametric approach is that the model we choose will usually not match the true unknown form of  $f$ . If the chosen model is too far from the true  $f$ , then our estimate will be poor. We can try to address this problem by choosing *flexible* models that can fit many different possible functional forms for  $f$ . But in general, fitting a more flexible model requires estimating a greater number of parameters. These more complex models can lead to a phenomenon known as *overfitting* the data, which essentially means they follow the errors, or *noise*, too closely. These issues are discussed throughout this book.

Figure 2.4 shows an example of the parametric approach applied to the **Income** data from Figure 2.3. We have fit a linear model of the form

$$\text{income} \approx \beta_0 + \beta_1 \times \text{education} + \beta_2 \times \text{seniority}.$$



**FIGURE 2.5.** A smooth thin-plate spline fit to the **Income** data from Figure 2.3 is shown in yellow; the observations are displayed in red. Splines are discussed in Chapter 7.

Since we have assumed a linear relationship between the response and the two predictors, the entire fitting problem reduces to estimating  $\beta_0$ ,  $\beta_1$ , and  $\beta_2$ , which we do using least squares linear regression. Comparing Figure 2.3 to Figure 2.4, we can see that the linear fit given in Figure 2.4 is not quite right: the true  $f$  has some curvature that is not captured in the linear fit. However, the linear fit still appears to do a reasonable job of capturing the positive relationship between **years of education** and **income**, as well as the slightly less positive relationship between **seniority** and **income**. It may be that with such a small number of observations, this is the best we can do.

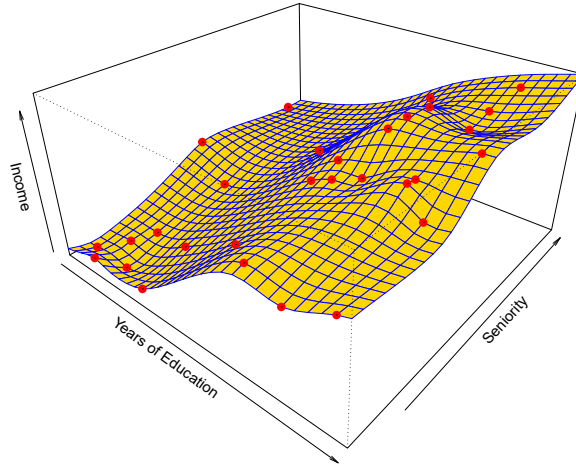
### Non-Parametric Methods

Non-parametric methods do not make explicit assumptions about the functional form of  $f$ . Instead they seek an estimate of  $f$  that gets as close to the data points as possible without being too rough or wiggly. Such approaches can have a major advantage over parametric approaches: by avoiding the assumption of a particular functional form for  $f$ , they have the potential to accurately fit a wider range of possible shapes for  $f$ . Any parametric approach brings with it the possibility that the functional form used to estimate  $f$  is very different from the true  $f$ , in which case the resulting model will not fit the data well. In contrast, non-parametric approaches completely avoid this danger, since essentially no assumption about the form of  $f$  is made. But non-parametric approaches do suffer from a major disadvantage: since they do not reduce the problem of estimating  $f$  to a small number of parameters, a very large number of observations (far more than is typically needed for a parametric approach) is required in order to obtain an accurate estimate for  $f$ .

An example of a non-parametric approach to fitting the **Income** data is shown in Figure 2.5. A *thin-plate spline* is used to estimate  $f$ . This approach does not impose any pre-specified model on  $f$ . It instead attempts

thin-plate  
spline





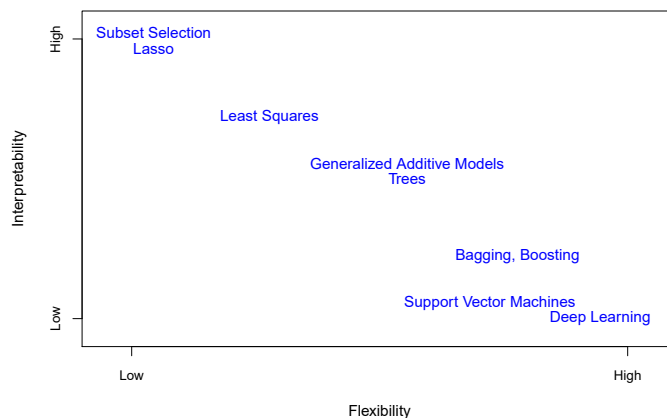
**FIGURE 2.6.** A rough thin-plate spline fit to the **Income** data from Figure 2.3. This fit makes zero errors on the training data.

to produce an estimate for  $f$  that is as close as possible to the observed data, subject to the fit—that is, the yellow surface in Figure 2.5—being *smooth*. In this case, the non-parametric fit has produced a remarkably accurate estimate of the true  $f$  shown in Figure 2.3. In order to fit a thin-plate spline, the data analyst must select a level of smoothness. Figure 2.6 shows the same thin-plate spline fit using a lower level of smoothness, allowing for a rougher fit. The resulting estimate fits the observed data perfectly! However, the spline fit shown in Figure 2.6 is far more variable than the true function  $f$ , from Figure 2.3. This is an example of overfitting the data, which we discussed previously. It is an undesirable situation because the fit obtained will not yield accurate estimates of the response on new observations that were not part of the original training data set. We discuss methods for choosing the *correct* amount of smoothness in Chapter 5. Splines are discussed in Chapter 7.

As we have seen, there are advantages and disadvantages to parametric and non-parametric methods for statistical learning. We explore both types of methods throughout this book.

### 2.1.3 The Trade-Off Between Prediction Accuracy and Model Interpretability

Of the many methods that we examine in this book, some are less flexible, or more restrictive, in the sense that they can produce just a relatively small range of shapes to estimate  $f$ . For example, linear regression is a relatively inflexible approach, because it can only generate linear functions such as the lines shown in Figure 2.1 or the plane shown in Figure 2.4. Other methods, such as the thin plate splines shown in Figures 2.5 and 2.6, are considerably more flexible because they can generate a much wider range of possible shapes to estimate  $f$ .



**FIGURE 2.7.** A representation of the tradeoff between flexibility and interpretability, using different statistical learning methods. In general, as the flexibility of a method increases, its interpretability decreases.

One might reasonably ask the following question: *why would we ever choose to use a more restrictive method instead of a very flexible approach?* There are several reasons that we might prefer a more restrictive model. If we are mainly interested in inference, then restrictive models are much more interpretable. For instance, when inference is the goal, the linear model may be a good choice since it will be quite easy to understand the relationship between  $Y$  and  $X_1, X_2, \dots, X_p$ . In contrast, very flexible approaches, such as the splines discussed in Chapter 7 and displayed in Figures 2.5 and 2.6, and the boosting methods discussed in Chapter 8, can lead to such complicated estimates of  $f$  that it is difficult to understand how any individual predictor is associated with the response.

Figure 2.7 provides an illustration of the trade-off between flexibility and interpretability for some of the methods that we cover in this book. Least squares linear regression, discussed in Chapter 3, is relatively inflexible but is quite interpretable. The *lasso*, discussed in Chapter 6, relies upon the linear model (2.4) but uses an alternative fitting procedure for estimating the coefficients  $\beta_0, \beta_1, \dots, \beta_p$ . The new procedure is more restrictive in estimating the coefficients, and sets a number of them to exactly zero. Hence in this sense the lasso is a less flexible approach than linear regression. It is also more interpretable than linear regression, because in the final model the response variable will only be related to a small subset of the predictors—namely, those with nonzero coefficient estimates. *Generalized additive models* (GAMs), discussed in Chapter 7, instead extend the linear model (2.4) to allow for certain non-linear relationships. Consequently, GAMs are more flexible than linear regression. They are also somewhat less interpretable than linear regression, because the relationship between each predictor and the response is now modeled using a curve. Finally, fully non-linear methods such as *bagging*, *boosting*, *support vector machines* with non-linear kernels, and *neural networks* (deep learning), discussed in Chapters 8, 9, and 10, are highly flexible approaches that are harder to interpret.

lasso

generalized  
additive  
modelbagging  
boosting  
support  
vector  
machine

We have established that when inference is the goal, there are clear advantages to using simple and relatively inflexible statistical learning methods. In some settings, however, we are only interested in prediction, and the interpretability of the predictive model is simply not of interest. For instance, if we seek to develop an algorithm to predict the price of a stock, our sole requirement for the algorithm is that it predict accurately—interpretability is not a concern. In this setting, we might expect that it will be best to use the most flexible model available. Surprisingly, this is not always the case! We will often obtain more accurate predictions using a less flexible method. This phenomenon, which may seem counterintuitive at first glance, has to do with the potential for overfitting in highly flexible methods. We saw an example of overfitting in Figure 2.6. We will discuss this very important concept further in Section 2.2 and throughout this book.

### 2.1.4 Supervised Versus Unsupervised Learning

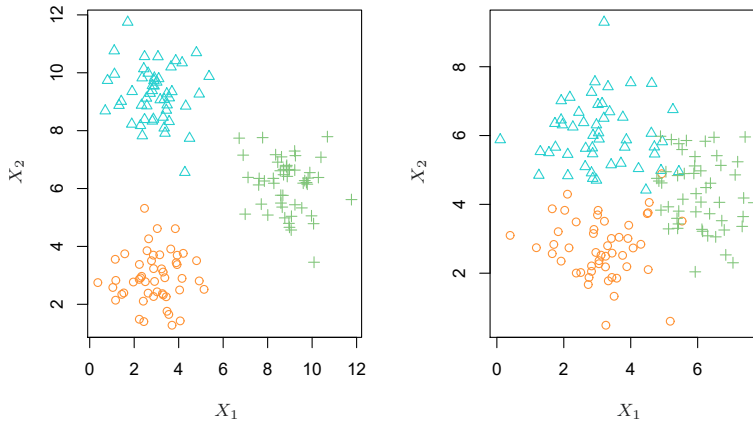
Most statistical learning problems fall into one of two categories: *supervised* or *unsupervised*. The examples that we have discussed so far in this chapter all fall into the supervised learning domain. For each observation of the predictor measurement(s)  $x_i$ ,  $i = 1, \dots, n$  there is an associated response measurement  $y_i$ . We wish to fit a model that relates the response to the predictors, with the aim of accurately predicting the response for future observations (prediction) or better understanding the relationship between the response and the predictors (inference). Many classical statistical learning methods such as linear regression and *logistic regression* (Chapter 4), as well as more modern approaches such as GAM, boosting, and support vector machines, operate in the supervised learning domain. The vast majority of this book is devoted to this setting.

supervised  
unsupervised

logistic  
regression

By contrast, unsupervised learning describes the somewhat more challenging situation in which for every observation  $i = 1, \dots, n$ , we observe a vector of measurements  $x_i$  but no associated response  $y_i$ . It is not possible to fit a linear regression model, since there is no response variable to predict. In this setting, we are in some sense working blind; the situation is referred to as *unsupervised* because we lack a response variable that can supervise our analysis. What sort of statistical analysis is possible? We can seek to understand the relationships between the variables or between the observations. One statistical learning tool that we may use in this setting is *cluster analysis*, or clustering. The goal of cluster analysis is to ascertain, on the basis of  $x_1, \dots, x_n$ , whether the observations fall into relatively distinct groups. For example, in a market segmentation study we might observe multiple characteristics (variables) for potential customers, such as zip code, family income, and shopping habits. We might believe that the customers fall into different groups, such as big spenders versus low spenders. If the information about each customer's spending patterns were available, then a supervised analysis would be possible. However, this information is not available—that is, we do not know whether each potential customer is a big spender or not. In this setting, we can try to cluster the customers on the basis of the variables measured, in order to identify

cluster  
analysis



**FIGURE 2.8.** A clustering data set involving three groups. Each group is shown using a different colored symbol. Left: The three groups are well-separated. In this setting, a clustering approach should successfully identify the three groups. Right: There is some overlap among the groups. Now the clustering task is more challenging.

distinct groups of potential customers. Identifying such groups can be of interest because it might be that the groups differ with respect to some property of interest, such as spending habits.

Figure 2.8 provides a simple illustration of the clustering problem. We have plotted 150 observations with measurements on two variables,  $X_1$  and  $X_2$ . Each observation corresponds to one of three distinct groups. For illustrative purposes, we have plotted the members of each group using different colors and symbols. However, in practice the group memberships are unknown, and the goal is to determine the group to which each observation belongs. In the left-hand panel of Figure 2.8, this is a relatively easy task because the groups are well-separated. By contrast, the right-hand panel illustrates a more challenging setting in which there is some overlap between the groups. A clustering method could not be expected to assign all of the overlapping points to their correct group (blue, green, or orange).

In the examples shown in Figure 2.8, there are only two variables, and so one can simply visually inspect the scatterplots of the observations in order to identify clusters. However, in practice, we often encounter data sets that contain many more than two variables. In this case, we cannot easily plot the observations. For instance, if there are  $p$  variables in our data set, then  $p(p-1)/2$  distinct scatterplots can be made, and visual inspection is simply not a viable way to identify clusters. For this reason, automated clustering methods are important. We discuss clustering and other unsupervised learning approaches in Chapter 12.

Many problems fall naturally into the supervised or unsupervised learning paradigms. However, sometimes the question of whether an analysis should be considered supervised or unsupervised is less clear-cut. For instance, suppose that we have a set of  $n$  observations. For  $m$  of the observations, where  $m < n$ , we have both predictor measurements and a response

measurement. For the remaining  $n - m$  observations, we have predictor measurements but no response measurement. Such a scenario can arise if the predictors can be measured relatively cheaply but the corresponding responses are much more expensive to collect. We refer to this setting as a *semi-supervised learning* problem. In this setting, we wish to use a statistical learning method that can incorporate the  $m$  observations for which response measurements are available as well as the  $n - m$  observations for which they are not. Although this is an interesting topic, it is beyond the scope of this book.

semi-  
supervised  
learning

### 2.1.5 Regression Versus Classification Problems

Variables can be characterized as either *quantitative* or *qualitative* (also known as *categorical*). Quantitative variables take on numerical values. Examples include a person's age, height, or income, the value of a house, and the price of a stock. In contrast, qualitative variables take on values in one of  $K$  different *classes*, or categories. Examples of qualitative variables include a person's marital status (married or not), the brand of product purchased (brand A, B, or C), whether a person defaults on a debt (yes or no), or a cancer diagnosis (Acute Myelogenous Leukemia, Acute Lymphoblastic Leukemia, or No Leukemia). We tend to refer to problems with a quantitative response as *regression* problems, while those involving a qualitative response are often referred to as *classification* problems. However, the distinction is not always that crisp. Least squares linear regression (Chapter 3) is used with a quantitative response, whereas logistic regression (Chapter 4) is typically used with a qualitative (two-class, or *binary*) response. Thus, despite its name, logistic regression is a classification method. But since it estimates class probabilities, it can be thought of as a regression method as well. Some statistical methods, such as  $K$ -nearest neighbors (Chapters 2 and 4) and boosting (Chapter 8), can be used in the case of either quantitative or qualitative responses.

quantitative  
qualitative  
categorical

class

regression  
classification

binary

We tend to select statistical learning methods on the basis of whether the response is quantitative or qualitative; i.e. we might use linear regression when quantitative and logistic regression when qualitative. However, whether the *predictors* are qualitative or quantitative is generally considered less important. Most of the statistical learning methods discussed in this book can be applied regardless of the predictor variable type, provided that any qualitative predictors are properly *coded* before the analysis is performed. This is discussed in Chapter 3.

## 2.2 Assessing Model Accuracy

One of the key aims of this book is to introduce the reader to a wide range of statistical learning methods that extend far beyond the standard linear regression approach. Why is it necessary to introduce so many different statistical learning approaches, rather than just a single *best* method? *There is no free lunch in statistics*: no one method dominates all others over all possible data sets. On a particular data set, one specific method may work

best, but some other method may work better on a similar but different data set. Hence it is an important task to decide for any given set of data which method produces the best results. Selecting the best approach can be one of the most challenging parts of performing statistical learning in practice.

In this section, we discuss some of the most important concepts that arise in selecting a statistical learning procedure for a specific data set. As the book progresses, we will explain how the concepts presented here can be applied in practice.

### 2.2.1 Measuring the Quality of Fit

In order to evaluate the performance of a statistical learning method on a given data set, we need some way to measure how well its predictions actually match the observed data. That is, we need to quantify the extent to which the predicted response value for a given observation is close to the true response value for that observation. In the regression setting, the most commonly-used measure is the *mean squared error* (MSE), given by

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2, \quad (2.5)$$

mean  
squared  
error

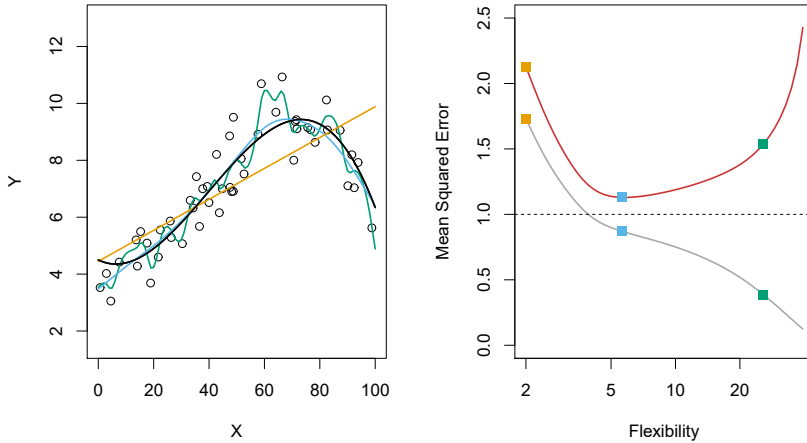
where  $\hat{f}(x_i)$  is the prediction that  $\hat{f}$  gives for the  $i$ th observation. The MSE will be small if the predicted responses are very close to the true responses, and will be large if for some of the observations, the predicted and true responses differ substantially.

The MSE in (2.5) is computed using the training data that was used to fit the model, and so should more accurately be referred to as the *training MSE*. But in general, we do not really care how well the method works on the training data. Rather, *we are interested in the accuracy of the predictions that we obtain when we apply our method to previously unseen test data*. Why is this what we care about? Suppose that we are interested in developing an algorithm to predict a stock's price based on previous stock returns. We can train the method using stock returns from the past 6 months. But we don't really care how well our method predicts last week's stock price. We instead care about how well it will predict tomorrow's price or next month's price. On a similar note, suppose that we have clinical measurements (e.g. weight, blood pressure, height, age, family history of disease) for a number of patients, as well as information about whether each patient has diabetes. We can use these patients to train a statistical learning method to predict risk of diabetes based on clinical measurements. In practice, we want this method to accurately predict diabetes risk for *future patients* based on their clinical measurements. We are not very interested in whether or not the method accurately predicts diabetes risk for patients used to train the model, since we already know which of those patients have diabetes.

training  
MSE

test data

To state it more mathematically, suppose that we fit our statistical learning method on our training observations  $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ , and we obtain the estimate  $\hat{f}$ . We can then compute  $\hat{f}(x_1), \hat{f}(x_2), \dots, \hat{f}(x_n)$ .



**FIGURE 2.9.** Left: Data simulated from  $f$ , shown in black. Three estimates of  $f$  are shown: the linear regression line (orange curve), and two smoothing spline fits (blue and green curves). Right: Training MSE (grey curve), test MSE (red curve), and minimum possible test MSE over all methods (dashed line). Squares represent the training and test MSEs for the three fits shown in the left-hand panel.

If these are approximately equal to  $y_1, y_2, \dots, y_n$ , then the training MSE given by (2.5) is small. However, we are really not interested in whether  $\hat{f}(x_i) \approx y_i$ ; instead, we want to know whether  $\hat{f}(x_0)$  is approximately equal to  $y_0$ , where  $(x_0, y_0)$  is a *previously unseen test observation not used to train the statistical learning method*. We want to choose the method that gives the lowest *test MSE*, as opposed to the lowest training MSE. In other words, if we had a large number of test observations, we could compute

$$\text{Ave}(y_0 - \hat{f}(x_0))^2, \quad (2.6)$$

the average squared prediction error for these test observations  $(x_0, y_0)$ . We'd like to select the model for which this quantity is as small as possible.

How can we go about trying to select a method that minimizes the test MSE? In some settings, we may have a test data set available—that is, we may have access to a set of observations that were not used to train the statistical learning method. We can then simply evaluate (2.6) on the test observations, and select the learning method for which the test MSE is smallest. But what if no test observations are available? In that case, one might imagine simply selecting a statistical learning method that minimizes the training MSE (2.5). This seems like it might be a sensible approach, since the training MSE and the test MSE appear to be closely related. Unfortunately, there is a fundamental problem with this strategy: there is no guarantee that the method with the lowest training MSE will also have the lowest test MSE. Roughly speaking, the problem is that many statistical methods specifically estimate coefficients so as to minimize the training set MSE. For these methods, the training set MSE can be quite small, but the test MSE is often much larger.

Figure 2.9 illustrates this phenomenon on a simple example. In the left-hand panel of Figure 2.9, we have generated observations from (2.1) with



the true  $f$  given by the black curve. The orange, blue and green curves illustrate three possible estimates for  $f$  obtained using methods with increasing levels of flexibility. The orange line is the linear regression fit, which is relatively inflexible. The blue and green curves were produced using *smoothing splines*, discussed in Chapter 7, with different levels of smoothness. It is clear that as the level of flexibility increases, the curves fit the observed data more closely. The green curve is the most flexible and matches the data very well; however, we observe that it fits the true  $f$  (shown in black) poorly because it is too wiggly. By adjusting the level of flexibility of the smoothing spline fit, we can produce many different fits to this data.

smoothing  
spline

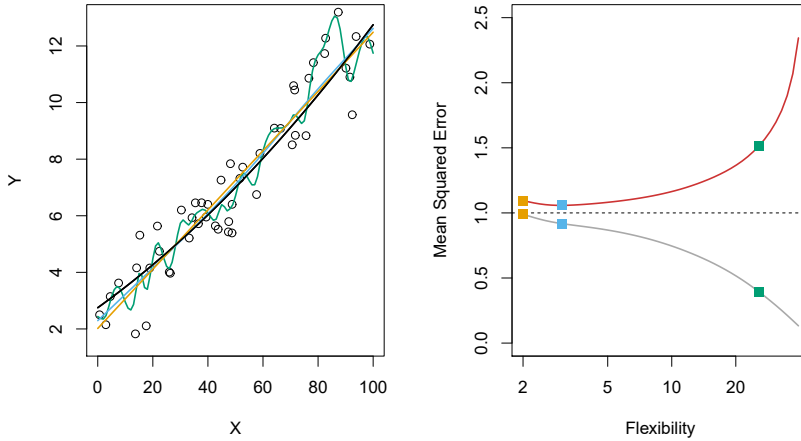
We now move on to the right-hand panel of Figure 2.9. The grey curve displays the average training MSE as a function of flexibility, or more formally the *degrees of freedom*, for a number of smoothing splines. The degrees of freedom is a quantity that summarizes the flexibility of a curve; it is discussed more fully in Chapter 7. The orange, blue and green squares indicate the MSEs associated with the corresponding curves in the left-hand panel. A more restricted and hence smoother curve has fewer degrees of freedom than a wiggly curve—note that in Figure 2.9, linear regression is at the most restrictive end, with two degrees of freedom. The training MSE declines monotonically as flexibility increases. In this example the true  $f$  is non-linear, and so the orange linear fit is not flexible enough to estimate  $f$  well. The green curve has the lowest training MSE of all three methods, since it corresponds to the most flexible of the three curves fit in the left-hand panel.

degrees of  
freedom

In this example, we know the true function  $f$ , and so we can also compute the test MSE over a very large test set, as a function of flexibility. (Of course, in general  $f$  is unknown, so this will not be possible.) The test MSE is displayed using the red curve in the right-hand panel of Figure 2.9. As with the training MSE, the test MSE initially declines as the level of flexibility increases. However, at some point the test MSE levels off and then starts to increase again. Consequently, the orange and green curves both have high test MSE. The blue curve minimizes the test MSE, which should not be surprising given that visually it appears to estimate  $f$  the best in the left-hand panel of Figure 2.9. The horizontal dashed line indicates  $\text{Var}(\epsilon)$ , the irreducible error in (2.3), which corresponds to the lowest achievable test MSE among all possible methods. Hence, the smoothing spline represented by the blue curve is close to optimal.

In the right-hand panel of Figure 2.9, as the flexibility of the statistical learning method increases, we observe a monotone decrease in the training MSE and a *U-shape* in the test MSE. This is a fundamental property of statistical learning that holds regardless of the particular data set at hand and regardless of the statistical method being used. As model flexibility increases, the training MSE will decrease, but the test MSE may not. When a given method yields a small training MSE but a large test MSE, we are said to be *overfitting* the data. This happens because our statistical learning procedure is working too hard to find patterns in the training data, and may be picking up some patterns that are just caused by random chance rather than by true properties of the unknown function  $f$ . When we overfit the training data, the test MSE will be very large because the supposed





**FIGURE 2.10.** Details are as in Figure 2.9, using a different true  $f$  that is much closer to linear. In this setting, linear regression provides a very good fit to the data.

patterns that the method found in the training data simply don't exist in the test data. Note that regardless of whether or not overfitting has occurred, we almost always expect the training MSE to be smaller than the test MSE because most statistical learning methods either directly or indirectly seek to minimize the training MSE. Overfitting refers specifically to the case in which a less flexible model would have yielded a smaller test MSE.

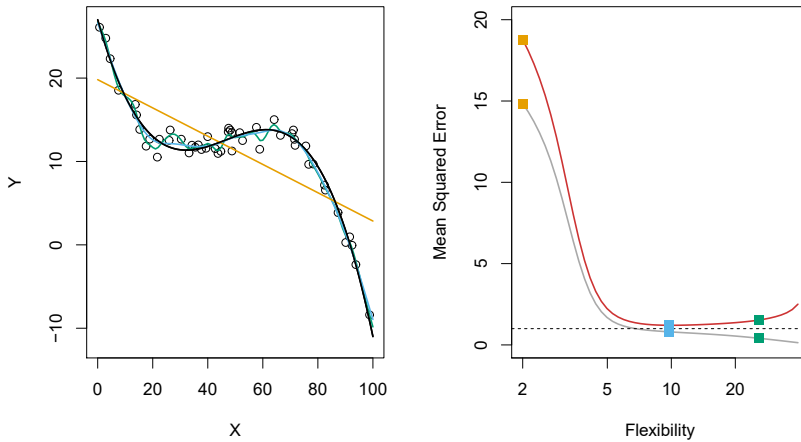
Figure 2.10 provides another example in which the true  $f$  is approximately linear. Again we observe that the training MSE decreases monotonically as the model flexibility increases, and that there is a U-shape in the test MSE. However, because the truth is close to linear, the test MSE only decreases slightly before increasing again, so that the orange least squares fit is substantially better than the highly flexible green curve. Finally, Figure 2.11 displays an example in which  $f$  is highly non-linear. The training and test MSE curves still exhibit the same general patterns, but now there is a rapid decrease in both curves before the test MSE starts to increase slowly.

In practice, one can usually compute the training MSE with relative ease, but estimating the test MSE is considerably more difficult because usually no test data are available. As the previous three examples illustrate, the flexibility level corresponding to the model with the minimal test MSE can vary considerably among data sets. Throughout this book, we discuss a variety of approaches that can be used in practice to estimate this minimum point. One important method is *cross-validation* (Chapter 5), which is a method for estimating the test MSE using the training data.

cross-validation

### 2.2.2 The Bias-Variance Trade-Off

The U-shape observed in the test MSE curves (Figures 2.9–2.11) turns out to be the result of two competing properties of statistical learning methods.



**FIGURE 2.11.** Details are as in Figure 2.9, using a different  $f$  that is far from linear. In this setting, linear regression provides a very poor fit to the data.

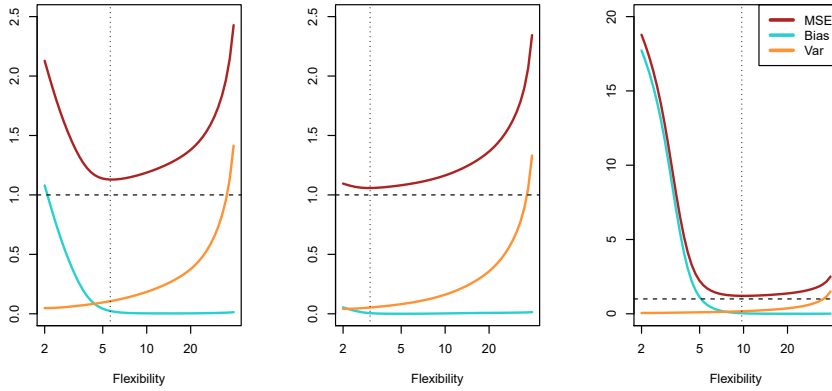
Though the mathematical proof is beyond the scope of this book, it is possible to show that the expected test MSE, for a given value  $x_0$ , can always be decomposed into the sum of three fundamental quantities: the *variance* of  $\hat{f}(x_0)$ , the squared *bias* of  $\hat{f}(x_0)$  and the variance of the error terms  $\epsilon$ . That is,

$$E \left( y_0 - \hat{f}(x_0) \right)^2 = \text{Var}(\hat{f}(x_0)) + [\text{Bias}(\hat{f}(x_0))]^2 + \text{Var}(\epsilon). \quad (2.7)$$

Here the notation  $E \left( y_0 - \hat{f}(x_0) \right)^2$  defines the *expected test MSE* at  $x_0$ , and refers to the average test MSE that we would obtain if we repeatedly estimated  $f$  using a large number of training sets, and tested each at  $x_0$ . The overall expected test MSE can be computed by averaging  $E \left( y_0 - \hat{f}(x_0) \right)^2$  over all possible values of  $x_0$  in the test set.

Equation 2.7 tells us that in order to minimize the expected test error, we need to select a statistical learning method that simultaneously achieves *low variance* and *low bias*. Note that variance is inherently a nonnegative quantity, and squared bias is also nonnegative. Hence, we see that the expected test MSE can never lie below  $\text{Var}(\epsilon)$ , the irreducible error from (2.3).

What do we mean by the *variance* and *bias* of a statistical learning method? *Variance* refers to the amount by which  $\hat{f}$  would change if we estimated it using a different training data set. Since the training data are used to fit the statistical learning method, different training data sets will result in a different  $\hat{f}$ . But ideally the estimate for  $f$  should not vary too much between training sets. However, if a method has high variance then small changes in the training data can result in large changes in  $\hat{f}$ . In general, more flexible statistical methods have higher variance. Consider the green and orange curves in Figure 2.9. The flexible green curve is following the observations very closely. It has high variance because changing any one of these data points may cause the estimate  $\hat{f}$  to change considerably.



**FIGURE 2.12.** Squared bias (blue curve), variance (orange curve),  $\text{Var}(\epsilon)$  (dashed line), and test MSE (red curve) for the three data sets in Figures 2.9–2.11. The vertical dotted line indicates the flexibility level corresponding to the smallest test MSE.

In contrast, the orange least squares line is relatively inflexible and has low variance, because moving any single observation will likely cause only a small shift in the position of the line.

On the other hand, *bias* refers to the error that is introduced by approximating a real-life problem, which may be extremely complicated, by a much simpler model. For example, linear regression assumes that there is a linear relationship between  $Y$  and  $X_1, X_2, \dots, X_p$ . It is unlikely that any real-life problem truly has such a simple linear relationship, and so performing linear regression will undoubtedly result in some bias in the estimate of  $f$ . In Figure 2.11, the true  $f$  is substantially non-linear, so no matter how many training observations we are given, it will not be possible to produce an accurate estimate using linear regression. In other words, linear regression results in high bias in this example. However, in Figure 2.10 the true  $f$  is very close to linear, and so given enough data, it should be possible for linear regression to produce an accurate estimate. Generally, more flexible methods result in less bias.

As a general rule, as we use more flexible methods, the variance will increase and the bias will decrease. The relative rate of change of these two quantities determines whether the test MSE increases or decreases. As we increase the flexibility of a class of methods, the bias tends to initially decrease faster than the variance increases. Consequently, the expected test MSE declines. However, at some point increasing flexibility has little impact on the bias but starts to significantly increase the variance. When this happens the test MSE increases. Note that we observed this pattern of decreasing test MSE followed by increasing test MSE in the right-hand panels of Figures 2.9–2.11.

The three plots in Figure 2.12 illustrate Equation 2.7 for the examples in Figures 2.9–2.11. In each case the blue solid curve represents the squared bias, for different levels of flexibility, while the orange curve corresponds to the variance. The horizontal dashed line represents  $\text{Var}(\epsilon)$ , the irreducible error. Finally, the red curve, corresponding to the test set MSE, is the sum

of these three quantities. In all three cases, the variance increases and the bias decreases as the method's flexibility increases. However, the flexibility level corresponding to the optimal test MSE differs considerably among the three data sets, because the squared bias and variance change at different rates in each of the data sets. In the left-hand panel of Figure 2.12, the bias initially decreases rapidly, resulting in an initial sharp decrease in the expected test MSE. On the other hand, in the center panel of Figure 2.12 the true  $f$  is close to linear, so there is only a small decrease in bias as flexibility increases, and the test MSE only declines slightly before increasing rapidly as the variance increases. Finally, in the right-hand panel of Figure 2.12, as flexibility increases, there is a dramatic decline in bias because the true  $f$  is very non-linear. There is also very little increase in variance as flexibility increases. Consequently, the test MSE declines substantially before experiencing a small increase as model flexibility increases.

The relationship between bias, variance, and test set MSE given in Equation 2.7 and displayed in Figure 2.12 is referred to as the *bias-variance trade-off*. Good test set performance of a statistical learning method requires low variance as well as low squared bias. This is referred to as a trade-off because it is easy to obtain a method with extremely low bias but high variance (for instance, by drawing a curve that passes through every single training observation) or a method with very low variance but high bias (by fitting a horizontal line to the data). The challenge lies in finding a method for which both the variance and the squared bias are low. This trade-off is one of the most important recurring themes in this book.

bias-variance  
trade-off

In a real-life situation in which  $f$  is unobserved, it is generally not possible to explicitly compute the test MSE, bias, or variance for a statistical learning method. Nevertheless, one should always keep the bias-variance trade-off in mind. In this book we explore methods that are extremely flexible and hence can essentially eliminate bias. However, this does not guarantee that they will outperform a much simpler method such as linear regression. To take an extreme example, suppose that the true  $f$  is linear. In this situation linear regression will have no bias, making it very hard for a more flexible method to compete. In contrast, if the true  $f$  is highly non-linear and we have an ample number of training observations, then we may do better using a highly flexible approach, as in Figure 2.11. In Chapter 5 we discuss cross-validation, which is a way to estimate the test MSE using the training data.

### 2.2.3 The Classification Setting

Thus far, our discussion of model accuracy has been focused on the regression setting. But many of the concepts that we have encountered, such as the bias-variance trade-off, transfer over to the classification setting with only some modifications due to the fact that  $y_i$  is no longer quantitative. Suppose that we seek to estimate  $f$  on the basis of training observations  $\{(x_1, y_1), \dots, (x_n, y_n)\}$ , where now  $y_1, \dots, y_n$  are qualitative. The most common approach for quantifying the accuracy of our estimate  $\hat{f}$  is the training *error rate*, the proportion of mistakes that are made if we apply

error rate

our estimate  $\hat{f}$  to the training observations:

$$\frac{1}{n} \sum_{i=1}^n I(y_i \neq \hat{y}_i). \quad (2.8)$$

Here  $\hat{y}_i$  is the predicted class label for the  $i$ th observation using  $\hat{f}$ . And  $I(y_i \neq \hat{y}_i)$  is an *indicator variable* that equals 1 if  $y_i \neq \hat{y}_i$  and zero if  $y_i = \hat{y}_i$ . If  $I(y_i \neq \hat{y}_i) = 0$  then the  $i$ th observation was classified correctly by our classification method; otherwise it was misclassified. Hence Equation 2.8 computes the fraction of incorrect classifications.

Equation 2.8 is referred to as the *training error* rate because it is computed based on the data that was used to train our classifier. As in the regression setting, we are most interested in the error rates that result from applying our classifier to test observations that were not used in training. The *test error* rate associated with a set of test observations of the form  $(x_0, y_0)$  is given by

$$\text{Ave}(I(y_0 \neq \hat{y}_0)), \quad (2.9)$$

where  $\hat{y}_0$  is the predicted class label that results from applying the classifier to the test observation with predictor  $x_0$ . A *good* classifier is one for which the test error (2.9) is smallest.

### The Bayes Classifier

It is possible to show (though the proof is outside of the scope of this book) that the test error rate given in (2.9) is minimized, on average, by a very simple classifier that *assigns each observation to the most likely class, given its predictor values*. In other words, we should simply assign a test observation with predictor vector  $x_0$  to the class  $j$  for which

$$\Pr(Y = j|X = x_0) \quad (2.10)$$

is largest. Note that (2.10) is a *conditional probability*: it is the probability that  $Y = j$ , given the observed predictor vector  $x_0$ . This very simple classifier is called the *Bayes classifier*. In a two-class problem where there are only two possible response values, say *class 1* or *class 2*, the Bayes classifier corresponds to predicting class one if  $\Pr(Y = 1|X = x_0) > 0.5$ , and class two otherwise.

Figure 2.13 provides an example using a simulated data set in a two-dimensional space consisting of predictors  $X_1$  and  $X_2$ . The orange and blue circles correspond to training observations that belong to two different classes. For each value of  $X_1$  and  $X_2$ , there is a different probability of the response being orange or blue. Since this is simulated data, we know how the data were generated and we can calculate the conditional probabilities for each value of  $X_1$  and  $X_2$ . The orange shaded region reflects the set of points for which  $\Pr(Y = \text{orange}|X)$  is greater than 50%, while the blue shaded region indicates the set of points for which the probability is below 50%. The purple dashed line represents the points where the probability is exactly 50%. This is called the *Bayes decision boundary*. The Bayes classifier's prediction is determined by the Bayes decision boundary; an observation that falls on the orange side of the boundary will be assigned

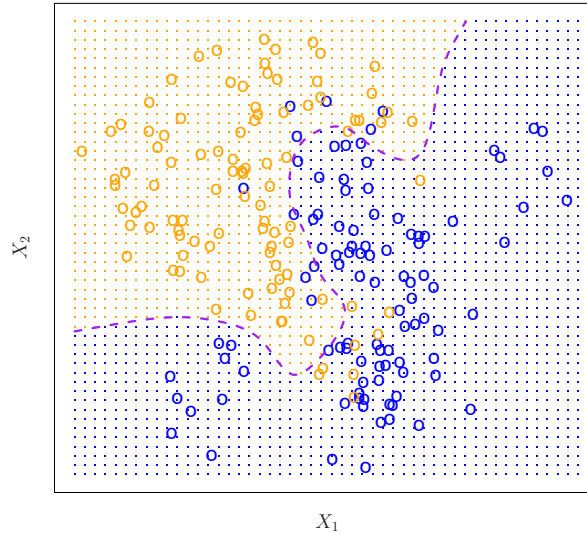
indicator  
variable

training  
error

test error

conditional  
probability  
Bayes  
classifier

Bayes  
decision  
boundary



**FIGURE 2.13.** A simulated data set consisting of 100 observations in each of two groups, indicated in blue and in orange. The purple dashed line represents the Bayes decision boundary. The orange background grid indicates the region in which a test observation will be assigned to the orange class, and the blue background grid indicates the region in which a test observation will be assigned to the blue class.

to the orange class, and similarly an observation on the blue side of the boundary will be assigned to the blue class.

The Bayes classifier produces the lowest possible test error rate, called the *Bayes error rate*. Since the Bayes classifier will always choose the class for which (2.10) is largest, the error rate will be  $1 - \max_j \Pr(Y = j|X = x_0)$  at  $X = x_0$ . In general, the overall Bayes error rate is given by

$$1 - E \left( \max_j \Pr(Y = j|X) \right), \quad (2.11)$$

where the expectation averages the probability over all possible values of  $X$ . For our simulated data, the Bayes error rate is 0.133. It is greater than zero, because the classes overlap in the true population, which implies that  $\max_j \Pr(Y = j|X = x_0) < 1$  for some values of  $x_0$ . The Bayes error rate is analogous to the irreducible error, discussed earlier.

### *K*-Nearest Neighbors

In theory we would always like to predict qualitative responses using the Bayes classifier. But for real data, we do not know the conditional distribution of  $Y$  given  $X$ , and so computing the Bayes classifier is impossible. Therefore, the Bayes classifier serves as an unattainable gold standard against which to compare other methods. Many approaches attempt to estimate the conditional distribution of  $Y$  given  $X$ , and then classify a given observation to the class with highest *estimated* probability. One such method is the *K-nearest neighbors* (KNN) classifier. Given a positive in-

*K*-nearest  
neighbors

teger  $K$  and a test observation  $x_0$ , the KNN classifier first identifies the  $K$  points in the training data that are closest to  $x_0$ , represented by  $\mathcal{N}_0$ . It then estimates the conditional probability for class  $j$  as the fraction of points in  $\mathcal{N}_0$  whose response values equal  $j$ :

$$\Pr(Y = j|X = x_0) = \frac{1}{K} \sum_{i \in \mathcal{N}_0} I(y_i = j). \quad (2.12)$$

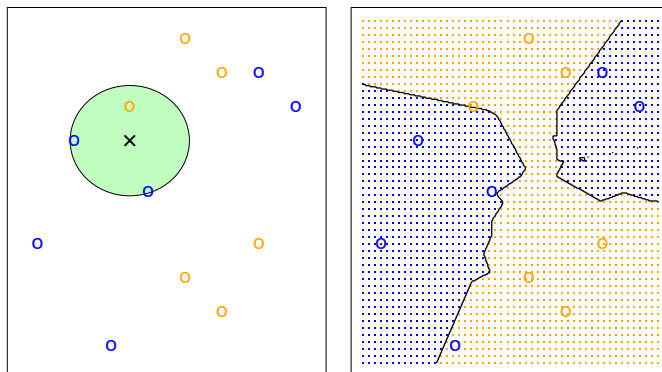
Finally, KNN classifies the test observation  $x_0$  to the class with the largest probability from (2.12).

Figure 2.14 provides an illustrative example of the KNN approach. In the left-hand panel, we have plotted a small training data set consisting of six blue and six orange observations. Our goal is to make a prediction for the point labeled by the black cross. Suppose that we choose  $K = 3$ . Then KNN will first identify the three observations that are closest to the cross. This neighborhood is shown as a circle. It consists of two blue points and one orange point, resulting in estimated probabilities of  $2/3$  for the blue class and  $1/3$  for the orange class. Hence KNN will predict that the black cross belongs to the blue class. In the right-hand panel of Figure 2.14 we have applied the KNN approach with  $K = 3$  at all of the possible values for  $X_1$  and  $X_2$ , and have drawn in the corresponding KNN decision boundary.

Despite the fact that it is a very simple approach, KNN can often produce classifiers that are surprisingly close to the optimal Bayes classifier. Figure 2.15 displays the KNN decision boundary, using  $K = 10$ , when applied to the larger simulated data set from Figure 2.13. Notice that even though the true distribution is not known by the KNN classifier, the KNN decision boundary is very close to that of the Bayes classifier. The test error rate using KNN is 0.1363, which is close to the Bayes error rate of 0.1304.

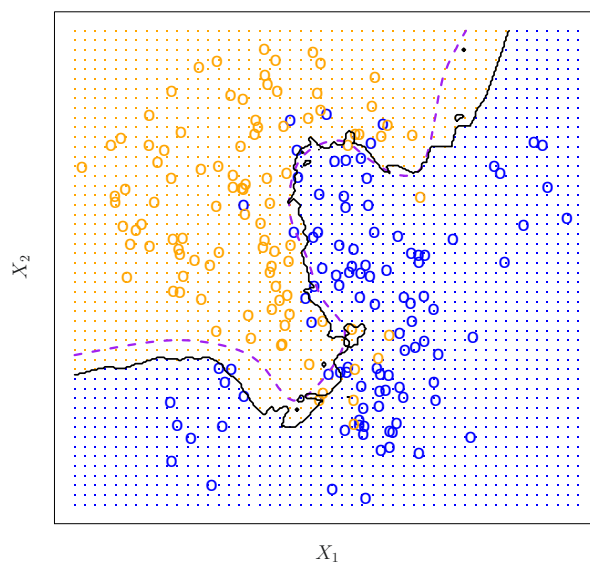
The choice of  $K$  has a drastic effect on the KNN classifier obtained. Figure 2.16 displays two KNN fits to the simulated data from Figure 2.13, using  $K = 1$  and  $K = 100$ . When  $K = 1$ , the decision boundary is overly flexible and finds patterns in the data that don't correspond to the Bayes decision boundary. This corresponds to a classifier that has low bias but very high variance. As  $K$  grows, the method becomes less flexible and produces a decision boundary that is close to linear. This corresponds to a low-variance but high-bias classifier. On this simulated data set, neither  $K = 1$  nor  $K = 100$  give good predictions: they have test error rates of 0.1695 and 0.1925, respectively.

Just as in the regression setting, there is not a strong relationship between the training error rate and the test error rate. With  $K = 1$ , the KNN training error rate is 0, but the test error rate may be quite high. In general, as we use more flexible classification methods, the training error rate will decline but the test error rate may not. In Figure 2.17, we have plotted the KNN test and training errors as a function of  $1/K$ . As  $1/K$  increases, the method becomes more flexible. As in the regression setting, the training error rate consistently declines as the flexibility increases. However, the test error exhibits a characteristic U-shape, declining at first (with a minimum at approximately  $K = 10$ ) before increasing again when the method becomes excessively flexible and overfits.



**FIGURE 2.14.** The KNN approach, using  $K = 3$ , is illustrated in a simple situation with six blue observations and six orange observations. Left: a test observation at which a predicted class label is desired is shown as a black cross. The three closest points to the test observation are identified, and it is predicted that the test observation belongs to the most commonly-occurring class, in this case blue. Right: The KNN decision boundary for this example is shown in black. The blue grid indicates the region in which a test observation will be assigned to the blue class, and the orange grid indicates the region in which it will be assigned to the orange class.

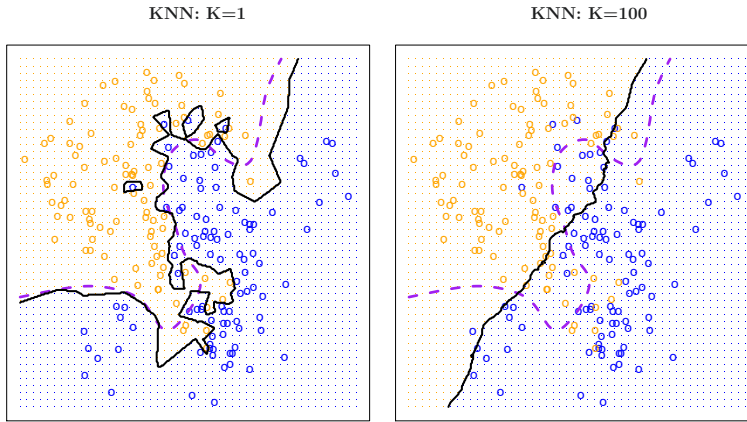
KNN: K=10



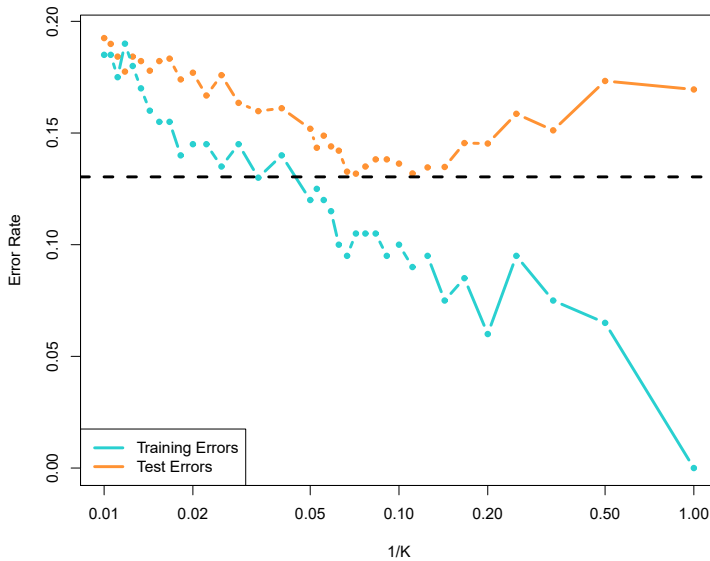
**FIGURE 2.15.** The black curve indicates the KNN decision boundary on the data from Figure 2.13, using  $K = 10$ . The Bayes decision boundary is shown as a purple dashed line. The KNN and Bayes decision boundaries are very similar.

In both the regression and classification settings, choosing the correct level of flexibility is critical to the success of any statistical learning method. The bias-variance tradeoff, and the resulting U-shape in the test error, can make this a difficult task. In Chapter 5, we return to this topic and discuss





**FIGURE 2.16.** A comparison of the KNN decision boundaries (solid black curves) obtained using  $K = 1$  and  $K = 100$  on the data from Figure 2.13. With  $K = 1$ , the decision boundary is overly flexible, while with  $K = 100$  it is not sufficiently flexible. The Bayes decision boundary is shown as a purple dashed line.



**FIGURE 2.17.** The KNN training error rate (blue, 200 observations) and test error rate (orange, 5,000 observations) on the data from Figure 2.13, as the level of flexibility (assessed using  $1/K$  on the log scale) increases, or equivalently as the number of neighbors  $K$  decreases. The black dashed line indicates the Bayes error rate. The jumpiness of the curves is due to the small size of the training data set.

various methods for estimating test error rates and thereby choosing the optimal level of flexibility for a given statistical learning method.

## 2.3 Lab: Introduction to Python

### 2.3.1 Getting Started

To run the labs in this book, you will need two things:

1. An installation of **Python3**, which is the specific version of **Python** used in the labs.
2. Access to **Jupyter**, a very popular **Python** interface that runs code through a file called a *notebook*.

notebook

You can download and install **Python3** by following the instructions available at [anaconda.com](https://anaconda.com).

There are a number of ways to get access to **Jupyter**. Here are just a few:

1. Using Google's **Colaboratory** service: [colab.research.google.com/](https://colab.research.google.com/).
2. Using **JupyterHub**, available at [jupyter.org/hub](https://jupyter.org/hub).
3. Using your own **jupyter** installation. Installation instructions are available at [jupyter.org/install](https://jupyter.org/install).

Please see the **Python** resources page on the book website [statlearning.com](https://statlearning.com) for up-to-date information about getting **Python** and **Jupyter** working on your computer.

You will need to install the **ISLP** package, which provides access to the datasets and custom-built functions that we provide. Inside a macOS or Linux terminal type `pip install ISLP`; this also installs most other packages needed in the labs. The **Python** resources page has a link to the **ISLP** documentation website.

To run this lab, download the file `Ch2-statlearn-lab.ipynb` from the **Python** resources page. Now run the following code at the command line: `jupyter lab Ch2-statlearn-lab.ipynb`.

If you're using Windows, you can use the **start menu** to access **anaconda**, and follow the links. For example, to install **ISLP** and run this lab, you can run the same code above in an **anaconda** shell.

### 2.3.2 Basic Commands

In this lab, we will introduce some simple **Python** commands. For more resources about **Python** in general, readers may want to consult the tutorial at [docs.python.org/3/tutorial/](https://docs.python.org/3/tutorial/).

Like most programming languages, **Python** uses *functions* to perform operations. To run a function called `fun`, we type `fun(input1,input2)`, where the inputs (or *arguments*) `input1` and `input2` tell **Python** how to run the function. A function can have any number of inputs. For example, the `print()` function outputs a text representation of all of its arguments to the console.

function

argument  
`print()`

```
In [1]: print('fit a model with', 11, 'variables')
```

fit a model with 11 variables

The following command will provide information about the `print()` function.

```
In [2]: print?
```

Adding two integers in `Python` is pretty intuitive.

```
In [3]: 3 + 5
```

```
Out[3]: 8
```

In `Python`, textual data is handled using *strings*. For instance, `"hello"` and `'hello'` are strings. We can concatenate them using the addition `+` symbol. string

```
In [4]: "hello" + " " + "world"
```

```
Out[4]: 'hello world'
```

A string is actually a type of *sequence*: this is a generic term for an ordered list. The three most important types of sequences are lists, tuples, and strings. We introduce lists now. sequence

The following command instructs `Python` to join together the numbers 3, 4, and 5, and to save them as a *list* named `x`. When we type `x`, it gives us back the list. list

```
In [5]: x = [3, 4, 5]
x
```

```
Out[5]: [3, 4, 5]
```

Note that we used the brackets `[]` to construct this list.

We will often want to add two sets of numbers together. It is reasonable to try the following code, though it will not produce the desired results.

```
In [6]: y = [4, 9, 7]
x + y
```

```
Out[6]: [3, 4, 5, 4, 9, 7]
```

The result may appear slightly counterintuitive: why did `Python` not add the entries of the lists element-by-element? In `Python`, lists hold *arbitrary* objects, and are added using *concatenation*. In fact, concatenation is the behavior that we saw earlier when we entered `"hello" + " " + "world"`. concatenation

This example reflects the fact that `Python` is a general-purpose programming language. Much of `Python`'s data-specific functionality comes from other packages, notably `numpy` and `pandas`. In the next section, we will introduce the `numpy` package. See [docs.scipy.org/doc/numpy/user/quickstart.html](https://docs.scipy.org/doc/numpy/user/quickstart.html) for more information about `numpy`.

### 2.3.3 Introduction to Numerical Python

As mentioned earlier, this book makes use of functionality that is contained in the **numpy** *library*, or *package*. A package is a collection of modules that are not necessarily included in the base **Python** distribution. The name **numpy** is an abbreviation for *numerical Python*. numpy  
package

To access **numpy**, we must first **import** it. import

```
In [7]: import numpy as np
```

In the previous line, we named the **numpy** *module* **np**; an abbreviation for easier referencing. module

In **numpy**, an *array* is a generic term for a multidimensional set of numbers. We use the **np.array()** function to define **x** and **y**, which are one-dimensional arrays, i.e. vectors. array  
np.array()

```
In [8]: x = np.array([3, 4, 5])
        y = np.array([4, 9, 7])
```

Note that if you forgot to run the **import numpy as np** command earlier, then you will encounter an error in calling the **np.array()** function in the previous line. The syntax **np.array()** indicates that the function being called is part of the **numpy** package, which we have abbreviated as **np**.

Since **x** and **y** have been defined using **np.array()**, we get a sensible result when we add them together. Compare this to our results in the previous section, when we tried to add two lists without using **numpy**.

```
In [9]: x + y
```

```
Out[9]: array([ 7, 13, 12])
```

In **numpy**, matrices are typically represented as two-dimensional arrays, and vectors as one-dimensional arrays.<sup>1</sup> We can create a two-dimensional array as follows.

```
In [10]: x = np.array([[1, 2], [3, 4]])
        x
```

```
Out[10]: array([[1, 2],
               [3, 4]])
```

The object **x** has several *attributes*, or associated objects. To access an attribute of **x**, we type **x.attribute**, where we replace **attribute** with the name of the attribute. For instance, we can access the **ndim** attribute of **x** as follows. attribute  
ndim

```
In [11]: x.ndim
```

```
Out[11]: 2
```

The output indicates that **x** is a two-dimensional array. Similarly, **x.dtype** is the *data type* attribute of the object **x**. This indicates that **x** is comprised of 64-bit integers: data type

---

<sup>1</sup>While it is also possible to create matrices using **np.matrix()**, we will use **np.array()** throughout the labs in this book.